모델 서빙 및 AIOps 구성 개인 서브노트

SKALA 판교 9반 P282 김민정

내 결론
가장 크게 배운 것은 모델이 틀려도 API는 200을 돌려준다는 점이다. HAIC 드리프트 배치에서 윈도우 RMSE가 6.43이어도 응답은 200이었고, 재학습 로그가 `[OK]`로 끝났는데도 /predict는 옛 모델로 답했다. 조 프로젝트에서 내가 맡은 LSTM 평가에서도 드리프트 구간 RMSE(8.07)가 직전값을 그대로 쓰는 단순 규칙(3.57)보다 나빴다.
그래서 모델을 운영한다면 세 가지를 챙기려고 한다.

1. 성능은 RMSE 숫자 하나가 아니라 단순한 방법보다 나은지로 판단한다.
2. 기준값은 한 번 재고 정하지 않는다. 여러 번 재 보고, 틀렸을 때 드는 비용까지 따져서 정한다.
3. 재학습을 자동으로 돌리는 것보다 나쁜 모델이 못 나가게 막고, 실제 응답에서 버전이 바뀌었는지 확인하는 것이 먼저다.

1. 이해관계자 가치 (Pain Point)

첫 시간에 교수님이 "데이터는 업무의 결과"라고 하셨다. 업무가 바뀌면 데이터도 바뀌지만, 이미 학습한 모델은 과거 데이터의 패턴을 기준으로 예측한다. 실습에서는 이 문제가 서빙, 버전 관리, 모니터링 세 군데에서 드러났다.

운영 체계가 없을 때	일어나는 상황	불편한 사람	실제로
서빙이 없다	모델이 노트북의 `.keras` 파일로만 남아 있으면 다른 시스템에서 호출할 수 없다. 호출할 수 있더라도 응답이 늦으면 사용하기 어렵다.	사용자, 서비스 운영자	[HAIC] 모델 없이 Lazy 서버를 띄운 뒤 /predict를 호출했을 때 3.09초 후 500 응답이 나왔다. `/health`는 계속 ok였지만 실제 예측 요청은 처리하지 못했다.
기록과 게이트가 없다	현재 서비스 중인 모델과 학습 시점과 데이터를 확인하기 어렵다. 성능이 더 나쁜 모델의 배포를 막을 기준도 없다.	모델 개발자, 운영자	[EtchGuard] 같은 모델을 게이트 0으로 다시 등록하자 `GATE FAILED`가 나왔고, 이 버전(v4)은 기록만 남고 Production(v3)은 그대로였다. 게이트가 없었다면 v4가 배포됐을 것이다.
감시가 없다	코드 오류가 없어도 예측 성능은 떨어질 수 있다. 사용자가 결과에 문제가 있다고 알려주기 전에는 확인하기 어렵다.	현업 사용자, 경영진	[HAIC] 드리프트 배치의 윈도우 RMSE가 6.43이어도 HTTP 200이 반환됐다. [EtchGuard] LSTM도 RMSE가 정상 구간 3.08에서 드리프트 구간 8.07로 올랐지만 API는 계속 200이었다.

조 프로젝트에 적용했을 때

EtchGuard는 식각 장비 5대의 최근 20사이클(10시간) 센서 12종으로 **30분 뒤 공정의 이상 점수(0~100)**를 예측한다. 같은 세 가지 문제가 여기서는 설비 엔지니어의 점검 시기, 품질과 수율 담당의 불량 확산, ML 엔지니어의 모

델 교체 판단으로 이어졌다. 나는 이 중 LSTM 학습과 정상 구간, 드리프트 구간 평가를 맡았다.

> **내 생각**
> 일반 소프트웨어는 버그가 나면 오류로 드러나지만, AI 서비스는 응답이 정상이어도 예측이 틀릴 수 있다. 특히 재학습 로그가 `[OK]`로 끝났는데 /predict가 이전 값(194.21)을 그대로 돌려준 일(4장 T3)이 오래 기억에 남는다. 그 뒤로는 로그보다 실제 응답을 먼저 확인한다.

---

2. 이를 해결하기 위한 AI 솔루션

서빙은 모델을 쓸 수 있게, MLOps는 믿고 바꿀 수 있게, AIOps는 나빠져도 회복하게 만든다. 실습에서는 각각 모델 호출, 학습 이력과 배포 기준 관리, 성능 모니터링과 재학습을 맡았다.

- **서빙**: 모델에 URL과 입력 규칙을 정해 다른 시스템에서도 정해진 형식으로 호출할 수 있게 한다.
- **MLOps**: 언제 어떤 데이터로 학습했는지 기록하고, 배포 게이트를 통과한 버전만 서비스에 적용한다.
- **AIOps**: 예측값과 실제값의 차이를 계속 확인하고, 성능이 나빠지면 알림과 재학습을 진행한다. 재학습한 모델도 게이트를 통과해야 서비스에 적용한다.

계층	막는 문제	핵심 기능	[HAIC] 담당 위치
서빙 (FastAPI)	노트북 속 모델	/predict, Pydantic 입력 검증 (422), /health, Lazy/Eager 로딩	`main.py`, `schemas.py`, `model_loader.py`, `routers/predict.py`
MLOps (MLflow, Docker)	기록 없음, 나쁜 모델 배포	run, param, metric 기록, Registry 등록, RMSE 게이트, Production 승격, 컨테이너 이미지	`train_and_register.py`, `Dockerfile`, `docker-compose.yml`
AIOps (모니터링, 재학습)	오류 없이 예측 성능이 떨어지는 모델	(예측, 실제) 누적, 윈도우 RMSE 판정, 알림, warm start 재학습, 게이트 재검증, 재배포	`drift_detector.py`, `retrain_trigger.py`, `predict.batch_test()`

3일 흐름 한눈에 보기

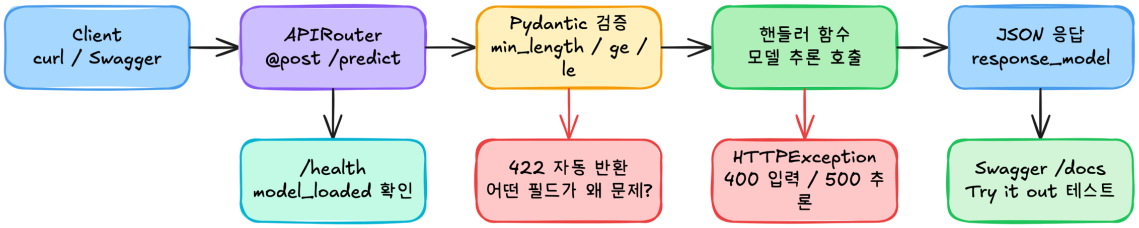
수업 내용을 Day별로 시각화해보면 이해에 도움이 될 것이라고 생각해서 excalidraw로 정리했다.

Day 1은 요청 하나가 서버 안에서 처리되는 과정이고, Day 2와 Day 3는 배포에서 자동 복구까지 이어지는 과정이다.

Day 1 · 모델 서빙 기초 (FastAPI)

학습 = 1회성 배치 / 서빙 = 24시간 상시 응답 (지연시간·가용성)

1. 요청 한 건이 처리되는 흐름



2. 동시 요청 처리 (동기 vs 비동기)

동기 def
요청 3건 순차 처리
총 약 3초

비동기 async def
run_in_threadpool
동시 처리 약 1초

Batching: 8건/50ms 모아 한 번에 추론
→ 처리량(Throughput) 향상

비동기 = 더 빠른 추론이 아니라 더 많은 동시 요청

3. 모델 로딩 전략 (Lazy vs Eager)

Lazy Loading
첫 요청 시 로드
시작 빠름 / 첫 요청 느림

Eager Loading
서버 시작 시 로드
시작 느림 / 첫 요청 빠름

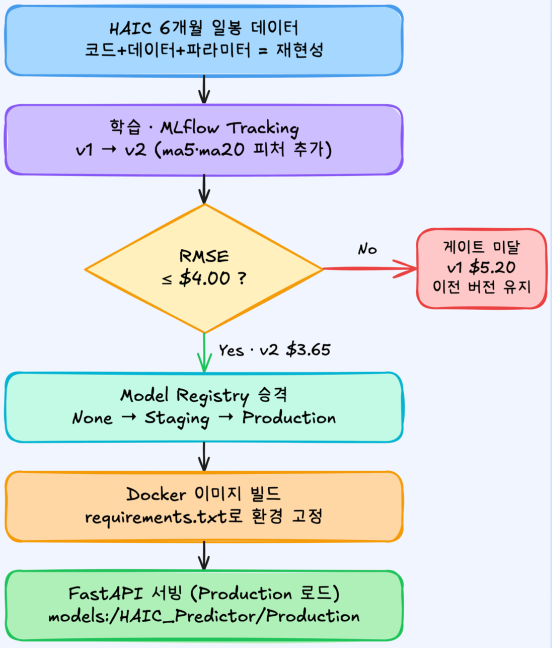
컨테이너 배포는 Eager 선호
모델 파일 문제를 기동 시점에 바로 발견

환경변수 LOADING_MODE=eager 로 전환

Day 3로 이어지는 기반: /health 헬스체크 + 요청·응답 로깅
→ 모니터링·AIOps의 감시 대상 및 원천 데이터

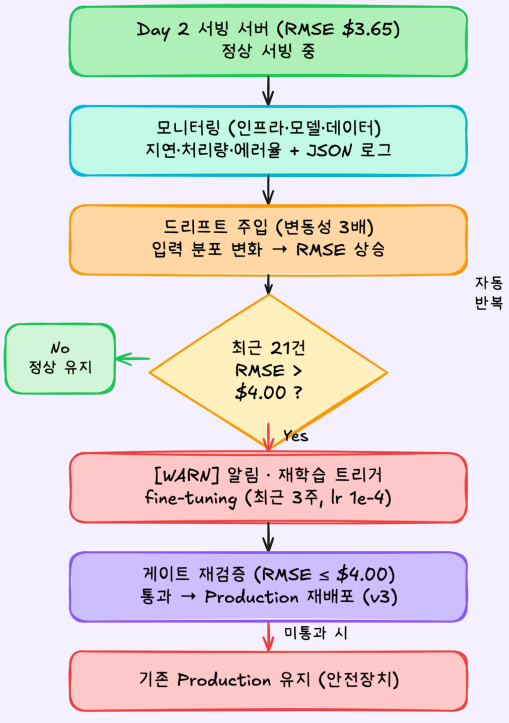
Day 2 MLOps → Day 3 AIOps : 배포부터 자동 복구까지

Day 2 · MLOps : 믿고 바꾸고 배포하기



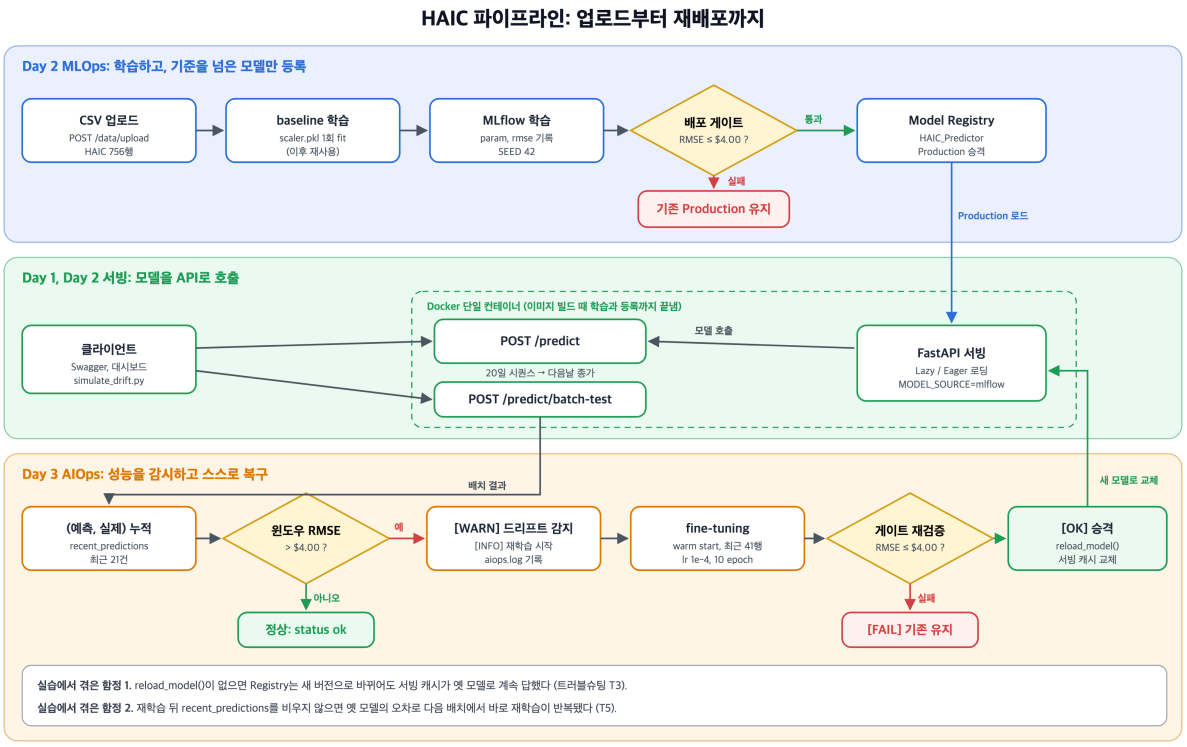
게이트 = 사람 판단 없이 수치로 자동 결정
실패해도 서비스는 이전 Production으로 계속

Day 3 · AIOps : 나빠져도 스스로 회복하기



3. 아키텍처 구성도

3-1. [HAIC] 업로드부터 재배포까지



- **학습과 등록 (Day 2)**: CSV를 올려 학습하고, RMSE가 $4.00 이하인 모델만 Registry의 Production으로 올린다.
- **서빙 (Day 1, Day 2)**: 컨테이너 안의 FastAPI가 Production 모델을 불러와 /predict와 /predict/batch-test에 응답한다.
- **감시와 복구 (Day 3)**: 최근 21건의 RMSE가 $4.00을 넘으면 재학습하고, 게이트를 다시 통과한 모델만 서빙 모델로 바꾼다.

3-2. 핵심 질문

1) Lazy vs Eager: 시작 시간과 첫 요청 시간

[HAIC] 같은 PC에서 프로세스를 새로 띄워 3회씩 측정 (시작 = 프로세스 실행 ~ 첫 /health 응답)

모드	서버 시작	첫 /predict	두 번째 /predict	/health (요청 전)
Lazy	**0.26초** (0.36 / 0.21 / 0.22)	2.31초 (2.74 / 2.19 / 2.00)	0.016초	model_loaded: false
Eager	**2.45초** (2.58 / 2.16 / 2.61)	0.153초 (0.146 / 0.164 / 0.150)	0.016초	model_loaded: true

조 프로젝트(EtchGuard)에서도 Lazy 첫 예측 2.60초, Eager 첫 예측 0.031초로 같은 경향이었다.

측정 결과에서 눈에 띈 점은 세 가지다.

1. **모델 로딩에는 두 방식 모두 약 2초가 걸렸다.** Lazy에서는 첫 요청을 처리할 때 모델을 불러오므로 사용자가 기다렸고, Eager에서는 서버 시작 과정에서 모델을 불러왔다.
2. **Eager도 첫 추론은 두 번째보다 오래 걸렸다.** [HAIC]에서 Eager 첫 요청은 0.153초, 두 번째는 0.016초로 약 10배 차이가 났다. TensorFlow가 첫 호출에서 실행 그래프를 준비하는 데 시간이 걸린 것으로 추정했다. 실제 서비스에 적용한다면 Eager 로딩 직후 더미 입력으로 한 번 추론하는 warm-up 단계를 추가하고 싶다.
3. **모델을 불러오지 못했을 때 오류가 발생하는 시점도 달랐다.** Day2 TODO를 채우기 전에는 Lazy 서버가 시작된 뒤 첫 요청에서 500을 반환했다. Eager 서버는 시작 단계에서 실패했다(exit code 3). 컨테이너 운영에서는 서버가 떠 있어도 예측을 처리하지 못하는 상태보다, 시작 단계에서 실패해 헬스체크로 바로 확인할 수 있는 편이 안전하다고 생각했다.

> **로딩 방식 선택**
> 운영 컨테이너에서는 Eager와 `/ready` 헬스체크를 함께 사용하려고 한다. 개발 중 `--reload`로 자주 재시작할 때는 Lazy를 사용하고, 장비별 모델이 수십 개지만 일부만 자주 사용하는 서비스라면 메모리를 고려해 Lazy와 캐시를 선택할 수 있다.

2) 학습 시점과 서빙 시점의 일치: 왜 20개 시퀀스이고, 왜 scaler를 다시 fit하지 않는가

- **20개 시퀀스**: LSTM은 최근 20거래일의 흐름을 입력으로 학습했다. 입력 shape는 [HAIC] (20, 2), [EtchGuard] (20, 12)로 고정되어 있다. 따라서 서빙에서도 학습할 때와 같은 형태로 입력을 받아야 한다. `Field(min_length=20, max_length=20)`으로 이 조건을 API에 반영했고, 실제로 19개를 보내면 422로 거절되는 것을 확인했다(5-2 Day1 캡처).
- **scaler를 다시 fit하지 않는 이유**: min-max 스케일러는 값을 0~1 범위로 변환한다. 모델 가중치는 Day1에 정한 변환 기준으로 학습됐다. 재학습할 때 새 데이터로 스케일러를 다시 fit하면 같은 가격도 다른 값으로 변환되어 기존 가중치와 맞지 않게 된다. 따라서 기존 가중치에서 이어서 학습하는 warm start에서는 같은 스케일러를 유지해야 한다고 이해했다.
- **실습에서 생각해 본 한계**: HAIC 데이터는 2008년 폭락처럼 가격대가 크게 달라지는 구간이 있다. 학습 범위를 벗어난 값은 변환 후에도 0~1 밖에 놓인다. 이런 상태가 오래 이어진다면 fine-tuning만 할 것이 아니라 스케일러를 다시 정하는 **새 base 학습**이 필요하다고 생각했다. 기존 스케일러로 재학습하는 것과 스케일러까지 바꾸어 다시 설계하는 것은 구분해서 결정해야 한다.
- **EtchGuard에서 수정한 부분**: HAIC는 scaler.pkl을 로컬 파일에서 읽기 때문에 다른 PC나 컨테이너에서 모델과 스케일러의 조합이 달라질 수 있었다. EtchGuard에서는 `scaler.json`을 모델과 같은 번들로 묶어 MLflow의 한 버전으로 등록했다. 이렇게 하면 모델을 불러올 때 함께 사용한 스케일러도 가져올 수 있다.

3) 배포 게이트: 없으면 무슨 일이 생기고, 통과 못 한 버전은 어떻게 되는가

- **게이트가 없다면**: Day3에서 재학습이 자동으로 실행되면 사람의 검토 없이 Production이 바뀔 수 있다. 실습에서는 **게이트가 새 모델을 배포해도 되는지 확인하는 유일한 단계**였으므로, 자동 재학습에도 배포 전 검증이 필요하다고 생각했다.
- **게이트를 통과하지 못한 버전**: Registry에는 기록을 남기되 Production 표시는 옮기지 않는다. [EtchGuard]에서 게이트를 0으로 낮춰 일부러 실패시키자 `[GATE FAILED] failed=['rmse'] rmse=3.164 → 기존 Production 유지`가 남았고, `registry list`에서도 v4는 `gate_passed=False`, Production은 v3 그대로였다. 6월 돌발 이상 때문에 시작된 재학습 모델(v2)도 HIGH Recall 0.00으로 게이트에서 막혀 v1이 계속 서비스했다.
- **운영에서 필요한 이유**: 기존 모델이 계속 서비스하므로 중단 없이 새 모델을 검증할 수 있다. 어떤 모델이 어떤 이유로 배포되지 않았는지도 기록으로 확인할 수 있다.
- **[HAIC] 결과**: base 학습 `[GATE PASSED] rmse=2.03 → HAIC_Predictor v1 promoted to Production`.

내 생각
실습을 해보니 절대 기준($4.00)만으로 배포 여부를 판단하기에는 부족하다고 느꼈다. HAIC의 fine-tuning RMSE는 1.47로 base(2.03)보다 낮았지만, 41행으로 만든 시퀀스를 8:2로 나누면 검증 표본이 5개 남짓이다. 표본이 적어 우연히 낮은 값이 나왔을 가능성도 고려해야 한다. 강의 자료의 "성능 회귀 테스트"처럼 **이전 버전보다 성능이 나빠지지 않았는지**도 확인할 필요가 있었다. 그래서 EtchGuard의 재학습 게이트에는 현재 Production보다 성능이 좋아야 한다는 Champion 비교 조건과 고정 정상셋 2,410건의 RMSE ≤ 4.0 조건을 추가했다.

4) 드리프트 판정 기준: 윈도우와 임계값을 바꾸면 오탐과 미탐이 어떻게 달라지는가

[HAIC] 실제 관측값 (윈도우 21건, 임계 $4.00): 정상 배치 윈도우 RMSE **2.97** → ok / 드리프트 배치 **6.43, 6.16, 4.94, 6.55, 5.13, 6.38** → 모두 재학습 트리거.

[HAIC] 10회 반복 측정. 랜덤워크라 실행마다 값이 달라서, `simulate_drift.py`의 배치 생성 함수를 그대로 쓰고 정상 배치와 드리프트 배치를 번갈아 10회씩 보냈다(2026-10-01, `project/` 복사본에서 실행).

배치	10회 윈도우 RMSE	최소 / 중앙 / 최대	판정
정상 (일별 변동성 1.2%)	1.87, 2.78, 3.50, 2.23, 3.01, 2.83, 3.35, 1.96, 1.97, 3.21	1.87 / 2.80 / 3.50	ok 10/10
드리프트 (변동성 3배)	6.49, 7.08, 6.63, 4.24, 6.91, 6.54, 6.93, 6.06, 7.30, 6.57	4.24 / 6.60 / 7.30	retrain_triggered 10/10

정상 최대(3.50)와 드리프트 최소(4.24) 사이의 여유는 0.74밖에 없었다. 임계값 $4.00은 그 사이에 있어 10회 모두 맞게 구분했지만, $5.00으로 올리면 4.24 배치를 놓치고(미탐 1/10), $3.50 근처로 내리면 정상 배치도 경보에 걸리기 시작한다. 한 번의 측정값만 보고 임계값을 정하면 위험하다는 것을 확인했고, 조 프로젝트에서 정상 구간 전체로 기준을 다시 잰 것도 같은 이유였다.

바꾸는 것	오탐 (정상인데 경보 발생)	미탐 (이상을 감지하지 못함)	내 해석
윈도우를 짧게	늘어남	줄어듦, 반응 빠름	몇 건의 큰 오차만으로도 판정 결과가 달라질 수 있다.
윈도우를 길게	줄어듦	반응이 늦어짐	이전 정상 구간의 오차가 함께 계산되어 드리프트가 늦게 드러난다.
임계값을 낮게	늘어남	줄어듦	정상 구간의 노이즈에도 재학습이 실행될 수 있다.
임계값을 높게	줄어듦	늘어남	실제 드리프트를 늦게 감지하거나 일부를 놓칠 수 있다.

[EtchGuard] 2025년 정상 구간(약 200일 × 5대)과 드리프트 구간에서 기준을 바꿔 본 결과

윈도우	임계값	연속	정상 오경보 (장비당 월 횟수)	드리프트 윈도우 판정률	탐지 지연
12시간	4.0	1회	2.57	84%	12h
12시간	4.0	2회	0.66	80%	24h
12시간	**4.0**	**3회 (채택)**	**0.03**	77%	36h
12시간	5.0	3회 (처음 기준)	0.00	61%	36h
12시간	6.0	3회	0.00	39%	36–48h
24시간	4.0	3회	0.00	75%	72h

장마와 고온 주간에는 ETCH-02의 윈도우 RMSE가 4.0을 두 번 연속 넘었지만(마지막 6.90) 3회에 못 미쳐 `watch`로만 표시됐다. 정상 구간에서 난 오경보 1회는 6월 ETCH-03의 돌발 이상(HIGH 17건)이었고, 이때 만든 재학습 모델은 게이트에서 막혀 기존 모델이 유지됐다.

내 생각
임계값은 통계만 보고 정할 수 없었다. 반도체 공정에서는 오경보로 점검을 한 번 더 하는 것보다 이상을 놓쳐 불량 웨이퍼가 계속 흘러가는 비용이 더 크다. 그렇다고 오경보가 잦으면 엔지니어가 알람을 무시하게 된다. 그래서 3회 연속 판정을 골랐고, 탐지가 24시간 늦어지는 대신 정상 구간 오경보가 월 2.57회에서 0.03회로 줄었다.
처음에는 경보 기준을 배포 기준(4.0)보다 높은 5.0으로 잡았다. 정상 노이즈 때문에 재학습이 도는 게 걱정됐기 때문이다. 그런데 표를 다시 보니 5.0에서는 드리프트 윈도우의 61%만 잡혔고, 4.0으로 내려도 오경보 목표(장비당 월 1회 미만)는 지켜졌다. 배포할 때는 4.0을 넘으면 안 된다고 해 놓고 운영 중에는 5.0까지 참는 것도 앞뒤가 맞지 않았다. 팀 논의 끝에 4.0으로 맞췄고, 숫자 하나를 고를 때도 처음 판단을 다시 확인해야 한다는 걸 배웠다.

5) 재학습 방식: 왜 scratch가 아니라 warm start인가, 게이트를 못 넘으면?

- **[HAIC] 데이터 양의 문제**: 최근 21거래일로 만들 수 있는 학습 시퀀스는 20개 남짓이다. 파라미터 약 1.6만 개인 LSTM을 처음부터 학습시키기에는 데이터가 부족하다고 판단했다.
- **[HAIC] 재학습에 쓰는 데이터**: 보낸 드리프트 배치가 아니라 업로드된 CSV의 마지막 41행(`21 + SEQ_LEN`)이다. 드리프트 배치는 경보를 울리는 신호 역할만 한다. 그래서 재학습 후 RMSE(1.44~1.69)는 "드리프트에 적응했다"는 뜻이 아니라 최근 41행 안의 검증 표본 5개 남짓에 대한 값이라고 읽어야 한다. EtchGuard에서는 감지 시점까지 실제로 확정된 최근 7일 데이터로 재학습하고 다음 1주(미사용 구간)로 따로 확인했다.
- **[EtchGuard] 같은 7일 데이터로 비교** (`scripts/compare_retrain_strategy.py`, 시드 3회 평균)

방식	최근 홀드아웃 RMSE	다음 1주 (학습에 사용하지 않은 미래 구간)	고정 정상셋
재학습 없음 (v1 유지)	7.858	7.295	3.105
warm start (lr 1e-4, 10 epoch)	2.649 ± 0.009	**3.561**	3.388
scratch (lr 1e-3, 30 epoch)	3.086 ± 0.359	4.025	6.427

- **결과를 보고 생각한 점**: scratch는 최근 데이터에 맞춰 학습하면서 기존 정상 구간의 성능이 떨어졌다(정상셋 6.43). 시드에 따라 결과도 ±0.36 정도 달라졌다. 기존 가중치에서 이어서 학습하는 warm start는 과거에 학습한 패턴을 유지하면서 최근 변화를 반영할 수 있어 결과가 더 안정적이었다(±0.009).
- **게이트를 통과하지 못하면**: 새 모델을 승격하지 않고 기존 Production이 계속 서비스하도록 한다. 이 구조에서는 승격을 보류하는 것으로 기존 버전을 유지하므로 자동 롤백 역할을 한다. [HAIC]는 `[FAIL] ... keep current production` 로그를 남기고, 승격 여부와 관계없이 `recent_predictions.clear()`로 판정 윈도우를 비운다. [EtchGuard]도 `monitor.reset()`으로 초기화하며, 이후 이상 상태가 새로 36시간 이상 지속되어야 다시 시도한다.
- **추가로 필요한 부분**: 지금 구현은 재학습이 연속으로 실패해도 로그만 남긴다. 실제 서비스에서는 N회 연속 실패했을 때 담당자에게 확인을 요청하는 에스컬레이션 규칙이 필요하다고 생각했다.

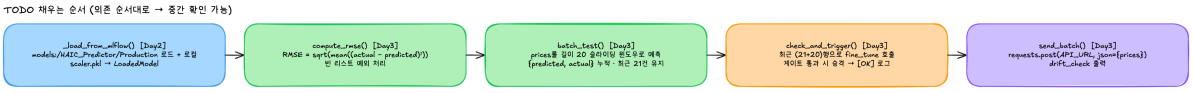
측정한 수치를 이해관계자 가치로 연결하면

측정값	누구에게	어떤 가치
첫 요청: Lazy 2.31초 vs Eager 0.153초 [HAIC] / 2.60초 vs 0.031초 [EtchGuard]	사용자, 설비 엔지니어, 운영자	배포 직후 첫 조회도 기다리지 않는다. 모델 파일 문제는 사용자가 아니라 기동 시점에 운영자가 발견한다.

측정값	누구에게	어떤 가치
잘못된 입력 422 (19개, close=0, 정답 필드 포함)	모델 개발자, 운영자	잘못된 입력이 예측값으로 둔갑하지 않고, loc와 msg로 원인을 바로 알 수 있다.
게이트: `GATE PASSED rmse=2.03` [HAIC], 실패한 v2와 v4는 기록만 남고 Production 유지 [EtchGuard]	운영자, 경영진	성능이 나쁜 모델이 자동으로 막히고, 왜 막혔는지 기록이 남는다.
드리프트 판정: 정상 1.87~3.50 vs 드리프트 4.24~7.30, 10/10 구분 [HAIC]	현업 사용자	틀린 예측을 계속 받기 전에 경보가 울린다. 정상 배치에서는 재학습이 일어나지 않아 비용이 낭비되지 않는다.
재학습: 감지에서 승격까지 5초, 응답 `production-v8` [HAIC] / 다음 주 RMSE 7.30 → 3.56, 4초 [EtchGuard]	사용자, ML 엔지니어	수작업 없이 성능이 회복되고, 응답의 버전으로 교체 여부를 확인할 수 있다.

TODO 채우기

TODO는 의존 순서대로 채웠다. 앞 함수가 끝나야 다음 함수를 중간에 확인할 수 있어서, 채운 순서를 그림으로 먼저 정리했다.



TODO	빠지면	근거
`model_loader._load_from_mlflow()`	MODEL_SOURCE=mlflow에서 Lazy는 첫 요청 500, Eager와 컨테이너는 기동 자체가 실패한다.	4-1 T2 (exit code 3)
`drift_detector.compute_rmse()`	드리프트를 숫자로 판단할 수 없다. 제곱을 빼면 +2와 −2가 상쇄돼 0.0, 제곱근을 빼면 단위가 달러²가 된다.	6장 빈칸 7
`routers/predict.batch_test()`	(예측, 실제) 쌍이 쌓이지 않아 판정 자체가 불가능하다. 실제값 인덱스를 한 칸만 당겨도 RMSE가 0.34로 낮아져 드리프트를 놓친다.	6장 빈칸 6
`retrain_trigger.check_and_trigger()`	감지해도 알림과 재학습이 없고 aiops.log 3단계 증거가 남지 않는다. `promoted` 조건이 없으면 게이트 탈락 모델도 `[OK]`로 기록된다.	6장 빈칸 11, 5-2 Day3
`simulate_drift.send_batch()`	서버 로직이 있어도 정상 상황과 드리프트 상황을 재현할 입력이 없어 임계값이 맞는지 검증할 수 없다.	3-2 4) 10회 측정

4. 트러블 슈팅

실습하면서 막혔던 문제를 증상, 원인, 해결, 전후 순서로 적고, 끝에 그 문제로 배운 점을 붙였다.

4-1. 트러블 슈팅 8건

T1. [HAIC Day1] 서버는 떴는데 /predict가 500

증상	`POST /predict` → `Internal Server Error` HTTP 500 (3.09초 소요). 같은 시점 `/health`는 `{"status":"ok","model_loaded":false,"loading_mode":"lazy"}`
원인	Lazy 모드에서는 모델 파일이 없어도 서버가 시작된다. 첫 요청을 처리할 때 `haic_v1.keras`를 불러오려다 실패했다.

해결	대시보드에서 CSV 업로드 → `python scripts/train_baseline_v1.py` (scaler fit on 756행, `haic_v1.keras`, `scaler.pkl` 생성)
전후	500 → 200 `{"predicted_close":192.54,"model_version":"v1-local"}`

`/health`에서 ok가 나와도 예측 서비스를 사용할 준비가 됐다는 뜻은 아니었다. 실습에서는 프로세스가 살아 있어도 모델이 로드되지 않아 예측이 실패했다. 조 프로젝트에서 `/health`(살아 있음)와 `/ready`(예측 가능, 미로딩 시 503)를 나눈 것도 이 때문이다.

T2. [HAIC Day2] MODEL_SOURCE=mlflow로 띄우자 NotImplementedError, Eager는 기동 실패

증상	Lazy: 첫 요청에서 `NotImplementedError: _load_from_mlflow를 구현하세요`. Eager: `ERROR: Application startup failed. Exiting.` (exit code 3)
원인	`model_loader._load_from_mlflow()`의 TODO를 아직 구현하지 않은 상태였다. Dockerfile은 `MODEL_SOURCE=mlflow`, `LOADING_MODE=eager`로 시작하므로 이 상태에서 빌드하면 컨테이너가 시작 직후 종료된다.
해결	`mlflow.tensorflow.load_model("models:/HAIC_Predictor/Production")` • 스케일러 로컬 로드 + 응답에 `production-v{N}` 버전 표기 (코드는 5-2 Day2)
전후	`{"predicted_close":194.21,"model_version":"production"}`, Eager 첫 요청 0.115초. `main.py`, `predict.py`는 한 줄도 수정하지 않음.

모델을 어디에서 가져올지 결정하는 코드를 함수 하나로 분리해 두었기 때문에, 로딩 소스를 바꾸어도 나머지 코드를 수정할 필요가 없었다.

T3. [HAIC Day3] 재학습 로그는 [OK]인데 /predict 결과가 그대로

증상	aiops.log에 `[OK] new_rmse=1.47 - production promoted: HAIC_Predictor v2`. 그런데 재학습 전후 /predict가 똑같이 `{"predicted_close":194.21,"model_version":"production"}`
원인 확인	v1과 v2를 각각 불러와 비교: v1 → 194.21, v2 → 195.19, `models:/HAIC_Predictor/Production` → 195.19. Registry는 바뀌었는데 서빙 프로세스의 `_model_cache`에 옛 모델이 남아 있었다.
해결	`reload_model()`을 추가해 승격 시 캐시를 새 모델로 교체, 응답 버전에 `production-v{N}`을 넣어 교체 여부를 눈으로 확인할 수 있게 함
전후	재학습 후 `{"predicted_close":195.16,"model_version":"production-v8"}`

재학습 로그에 성공이 찍혀도 서빙 중인 모델까지 바뀐 것은 아니었다. 응답에 버전을 넣은 뒤로는 교체 여부를 바로 확인할 수 있었다. 조 프로젝트에서는 **새 모델을 먼저 불러온 뒤 참조를 바꾸는** 방식으로 이 문제를 피했다. 빈칸 실습 정답 코드로 루프를 돌렸을 때도 `[OK] ... v13`인데 /predict는 `production`으로 같은 값이었다(6장).

T4. [HAIC Day3] Registry에 v1~v11이 전부 Production

증상	MLflow UI의 HAIC_Predictor 버전 목록에서 Version 1~11의 Stage가 모두 Production. 학습 때마다 `FutureWarning: transition_model_version_stage is deprecated since 2.9.0`
원인	`transition_model_version_stage()`를 `archive_existing_versions` 없이 호출해 이전 버전이 Production에 그대로 남음. 서빙은 `get_latest_versions()`가 최신 하나를 골라 줘서 정상 동작했고, 그래서 발견이 늦었다.
해결	HAIC에서는 `archive_existing_versions=True`를 설정하면 이전 Production 버전이 남는 것을 막을 수 있다. 조 프로젝트는 stage 대신 alias `@Production`을 사용했다. alias를 새 버전으로 옮기면 이전 버전에서는 해제된다. 롤백은 `python -m semiconductor.registry rollback --version N`으로 실행한다.

전후	[EtchGuard] `registry list` : `v1 base gate_passed=True` / `v2 fine-tune gate_passed=False` / `v3 fine-tune gate_passed=True ← Production` / `v4 base gate_passed=False` , Production이 정확히 하나

Registry에서 현재 서비스 버전을 확인했을 때 Production이 여러 개라면 어느 버전을 사용하는지 바로 알기 어려웠다. 예측 요청이 정상 처리되는 것과 버전 관리 상태가 올바른 것은 따로 확인해야 한다고 느꼈다.

T5. [HAIC Day3] 드리프트 배치마다 재학습이 연달아 일어남

증상	simulate를 5번 돌리자 30초 동안 v3 → v4 → v5 → v6 → v7 승격. `[WARN] drift detected - window_rmse=6.43 > 4.00` 이 배치마다 반복
원인	(1) 재학습 전 모델이 만든 오차가 `recent_predictions` 에 남으면 다음 판정에서 바로 다시 드리프트로 잡힌다. (2) 그 부분을 막아도, 변동성 3배 배치는 매번 실제로 드리프트이고 판정이 1회 초과 기준이라 매번 재학습한다.
해결	(1)은 재학습 시도 후 `recent_predictions.clear()` . (2)는 정책 문제로 보고 EtchGuard에서 3회 연속 판정 + 재학습 락(실행 중이면 `retrain_in_progress` 로 응답)으로 바꿈
전후	[EtchGuard] 장비 5대가 동시에 드리프트를 감지해도 재학습은 1회(ETCH-01이 트리거, 나머지 4대는 `retrain_in_progress`), 4초 만에 production-v1 → v3

처음에는 재학습이 반복되는 것을 코드 오류로 생각했지만, 확인해 보니 코드는 정해진 규칙대로 동작하고 있었다. 기존 오차를 지우는 수정과 별개로, 한 번만 임계값을 넘어도 재학습하는 정책을 바꿀 필요가 있었다.

T6. [EtchGuard] 입력 분포(PSI)로 경보를 걸자 정상 주의 100%가 경보

증상	고정 PSI 0.25 기준을 적용하면 정상 주간 70주 모두 경보 (`fixed_psi_0.25_alarm_rate_normal_weeks: 1.0`)
원인	장비 열화와 정비 주기 때문에 정상에도 챔버 압력, 진동, 파티클 PSI가 1 이상. 계절을 무시한 연간 통합 기준이면 외기 온도 PSI가 4.87까지 커진다(같은 달 기준은 0.28).
해결	경보 판정은 예측 오차(RMSE)로만 하고, PSI는 **전년도 같은 달 기준 + 평상시 대비 배율**로 바꿔 원인 후보 센서 순위에만 사용
전후	드리프트 경보에 `rf_power_w x10.2, vibration_mm_s x2.6, chamber_pressure_mtorr x2.4` 가 붙음. 데이터에 실제로 주입한 변화(RF↓, 진동↑, 압력↑)와 일치

가이드 부록의 "KS, PSI로 먼저 감지" 방식을 적용해 보니, 우리 도메인에서는 정상 구간에도 경보가 너무 자주 발생했다. 그래서 PSI를 경보 판정에 그대로 쓰기보다 원인 후보를 찾는 진단에 사용하기로 했다.

T7. [HAIC] 같은 코드인데 로컬과 컨테이너의 baseline RMSE가 다름

증상	`train_baseline_v1.py` : 로컬 RMSE 2.72, 컨테이너 빌드 중 3.01. 반면 `train_and_register.py` 는 양쪽 모두 2.03
원인	`train_and_register.py` 는 `keras.utils.set_random_seed(42)` 로 시드를 고정하지만 `train_baseline_v1.py` 에는 시드 고정이 없다.
해결	재현이 필요한 학습 스크립트는 모두 시드 고정. EtchGuard는 `set_random_seed(42)` • `enable_op_determinism()`
전후	[EtchGuard] macOS 3.164 vs 컨테이너(Linux ARM64) 3.165. 남은 0.001은 플랫폼 연산 차이

컨테이너로 라이브러리 버전을 맞추더라도 난수까지 같아지는 것은 아니었다. 같은 결과를 재현하려면 환경, 시드, 데이터를 함께 맞춰야 한다는 것을 확인했다. 플랫폼에 따른 작은 차이도 남았으므로 게이트 기준에는 약간의 여유가 필요하다고 생각했다.

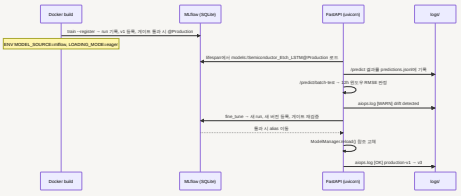
T8. [MLflow 실습] UI를 띄웠는데 실험 기록이 안 보임

증상	노트북에서 run은 끝났는데 `mlruns/` 폴더 아래에서 기록을 찾을 수 없음 (수업 중 교수님도 같은 현상을 짚으심)
원인	MLflow 3.x에서는 추적 서버의 기본 backend가 `mlflow.db` (SQLite)이고 아티팩트는 `mlartifacts/` 에 쌓인다. 폴더가 아니라 DB를 봐야 한다.
해결	같은 저장소를 바라보도록 실행: `mlflow server --backend-store-uri sqlite:///mlflow.db --default-artifact-root ./mlartifacts --host 127.0.0.1 --port 5001`
전후	diabetes_experiment1(GenAI), diabetes_experiment2(RandomForest) 실험과 trace가 모두 조회됨 (9장 캡처)

도구 버전이 바뀌면 기록이 저장되는 위치도 다시 확인해야 한다는 것을 알게 됐다. EtchGuard에서는 저장소 설정이 서로 달라지지 않도록 `MLFLOW_TRACKING_URI`를 `config.tracking_uri()` 한곳에서만 정했다.

4-2. 요소 기술 간 연결 정리 (FastAPI ↔ MLflow ↔ Docker ↔ 로그)

처음에는 FastAPI, MLflow, Docker, 로그를 각각 따로 사용하는 도구로 생각했다. EtchGuard 컨테이너가 시작되고 재학습이 실행되는 과정을 순서대로 따라가면서 각 도구가 어디에서 연결되는지 정리할 수 있었다.



용어 정리

용어	내가 이해한 뜻
Experiment / Run / Artifact	실험 묶음 / 학습 한 번의 기록 / 그 학습이 남긴 파일(모델, evaluation.json 등)
Stage vs Alias	Stage는 각 버전에 붙는 상태값이라 여러 버전이 동시에 Production일 수 있다. Alias는 같은 이름으로 하나의 버전만 가리킨다.
pyfunc / signature	프레임워크와 관계없이 `predict()` 하나로 부를 수 있게 감싼 MLflow 모델 형식 / 입력과 출력 shape와 타입 계약
데이터 드리프트 vs 컨셉 드리프트	입력 분포가 변함(가격 변동성 급증, 센서값 이동) vs 입력과 정답의 관계가 변함(같은 신호인데 결과가 달라짐)
Champion / Challenger	현재 서비스 중인 모델과 새 후보 모델을 같은 홀드아웃 데이터로 비교한다. 후보 모델의 성능이 더 좋아야 교체한다.
골든셋 (고정 정상셋)	재학습 때마다 동일하게 사용하는 정상 데이터다. 새 모델의 기존 정상 구간 성능이 떨어졌는지 확인한다.
Warm start	이미 학습된 가중치에서 낮은 학습률로 이어서 학습하는 것. 처음부터 다시 하는 scratch의 반대

5. 레퍼런스

5-1. 강의 키워드 요약

▼ Day1 모델 서빙

키워드	내 말로 한 줄
AIOps	운영(Operations)에 AI를 붙여, 사람이 지켜보던 지연, 에러, 성능 저하를 자동으로 감지하고 대응하게 하는 것
학습 vs 서빙	학습은 1회성 배치로 정확도를 평가하고, 서빙은 24시간 제공하면서 지연, 처리량, 가용성을 평가한다. 학습 실패는 다시 실행할 수 있지만 서빙 실패는 서비스 장애로 이어진다.
배치 vs 실시간 추론	정해진 시간에 몰아서(야간 리포트) vs 요청 즉시(이상 탐지). EtchGuard의 batch-test는 이름은 배치지만 확정 점수가 들어오는 피드백 경로다.
REST, gRPC, WebSocket, WebRTC	요청마다 연결(REST), 내부 고성능 통신(gRPC), 연결 유지(WebSocket), 연결 대상 확정 후 P2P 미디어(WebRTC)
동기 vs 비동기	비동기는 더 빠른 추론이 아니라 기다리는 동안 다른 요청을 받는 구조. 순서가 중요한 일은 동기로 묶는다.
Pydantic 스키마	요청과 응답 규격. 어긋나면 함수 실행 전에 422와 이유(loc, msg)를 돌려준다.
API Schema vs DB Model	검증용 객체는 요청마다 생성되고, DB 스키마는 저장되는 테이블 구조를 정의한다. 계층을 분리하면 한쪽의 변경이 다른 쪽에 미치는 영향을 줄일 수 있다.
Swagger(/docs)	타입 힌트와 스키마만으로 생성되는 API 계약서. 운영에서는 일부러 숨기기도 한다.
서빙 안정성	입력 검증, 에러 핸들링(HTTPException), 헬스체크, Lazy/Eager 로딩, 로깅

▼ Day2 MLOps

키워드	내 말로 한 줄
MLOps	DevOps 원칙에 데이터와 모델 성능까지 관리 대상으로 더한 것. 재현하려면 코드 + 데이터 + 하이퍼파라미터가 모두 같아야 한다.
성숙도 Level 0 → 1 → 2	노트북 수동 학습 → MLflow 기록, Registry, 게이트 → 새 데이터 자동 재학습과 모니터링 순서로 확장한다. Day1~3도 같은 순서로 진행했다.
MLflow 4요소	Tracking(실험 기록), Projects(코드 패키징), Models(모델 형식), Registry(버전과 상태 관리)
CI/CD for ML	새 데이터 → 자동 학습 → RMSE 게이트 → 승격 → 컨테이너 재배포로 이어진다. 게이트를 통과하지 못하면 이전 버전으로 서빙을 유지한다.
환경 일관성	로컬 RMSE 3.65가 서버에서 6.20이 되는 문제를 requirements 고정 + 컨테이너로 막는다.
데이터와 모델 검증	입력 스키마 검증, 이전 버전 대비 성능 회귀 테스트, 절대 기준 배포 게이트
MLOps vs AIOps	"어떻게 만들어 내보낼까"(모델 착수 ~ 배포) vs "지금도 잘 돌아가는가"(서빙 시작 이후 시스템 전체)

▼ Day3 AIOps

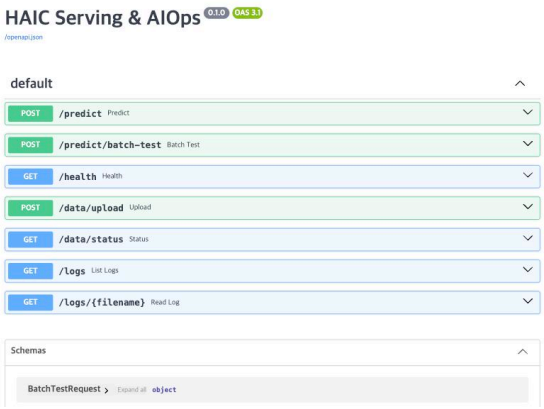
키워드	내 말로 한 줄
모니터링 3대 대상	인프라(컨테이너, /health), 모델(RMSE, 예측 분포, 지연), 데이터(입력 분포 변화)

키워드	내 말로 한 줄
추론 지표	지연시간(ms), 처리량(초당 요청), 에러율(5xx 비율). EtchGuard는 엔드포인트별 p50/p95로 수집한다.
구조화 로깅	집계 스크립트가 바로 읽을 수 있도록 JSON으로 기록한다. actual은 정답이 도착한 뒤 기록한다.
알림 설계	조건, 측정 주기, 대응 동작을 한 세트로 정의 (예: RMSE 4.00 초과, 최근 21건, 알림 + 재학습)
재학습 트리거 구조	이상 탐지 → 알림 → 최근 데이터 수집 → 재학습 → 게이트 재검증 → 재배포, 실패 시 기존 유지
드리프트 시뮬레이션	기준 통계 → 정상 입력으로 베이스라인 → 변동성 3배 데이터 생성 → 주입 → RMSE, 알림, 재학습 관찰

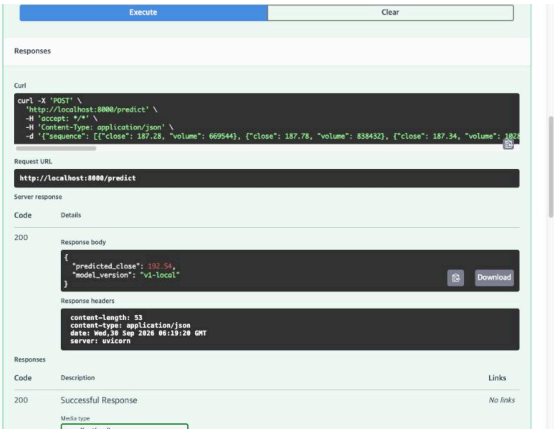
5-2. Day별 실행 증빙

Day1: HAIC baseline LSTM을 FastAPI로 서빙

1. /docs(Swagger)와 /predict Try it out

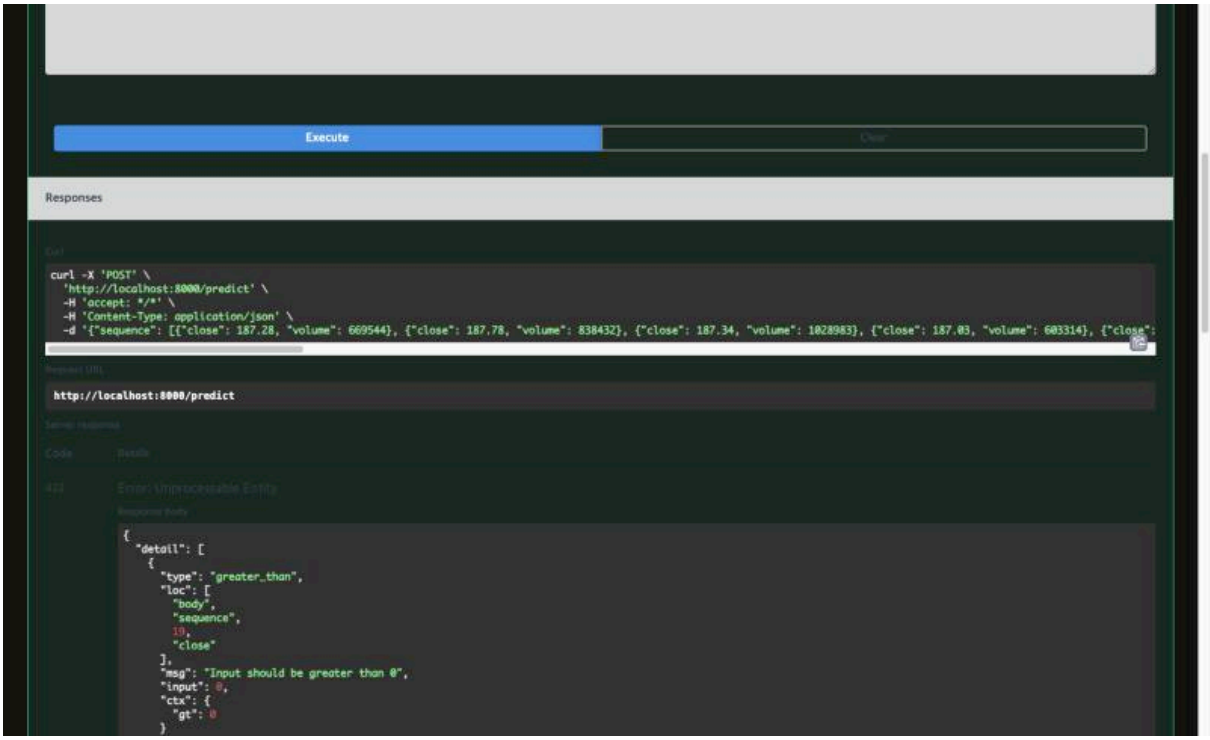


[HAIC] Swagger 전체 엔드포인트



[HAIC] /predict 200: predicted_close와 model_version

2. 잘못된 입력에 대한 422



[HAIC] 시퀀스 19개 → 422

```
# [HAIC] /predict 입력 검증 (day1_evidence/step4_predict_validation.txt)
20개 정상        → 200 {"predicted_close":192.54,"model_version":"v1-local"}  (실제 다음날 종가 192.71)
19개             → 422 too_short    "List should have at least 20 items after validation, not 19"
21개             → 422 too_long     "List should have at most 20 items after validation, not 21"
마지막 close = 0 → 422 greater_than loc=["body","sequence",19,"close"] "Input should be greater than 0"
close=-10.5, volume=-1 → 422 오류 2건 (greater_than, greater_than_equal)
빈 본문 {}       → 422 missing      loc=["body","sequence"]
```

3. /health의 Lazy/Eager 차이

```
[HAIC] Lazy   요청 전 {"status":"ok","model_loaded":false,"loading_mode":"lazy"}
              요청 후 {"status":"ok","model_loaded":true,"loading_mode":"lazy"}
              log: [lazy] model loaded in 2.643s on first request
[HAIC] Eager  요청 전 {"status":"ok","model_loaded":true,"loading_mode":"eager"}
              log: [eager] model loaded in 2.284s at startup
```

4. Lazy vs Eager 측정표: 3-2의 1번 표 참고 (HAIC 3회 원자료 아래)

```
lazy  run1 | start=0.36s | 1st=200 2.737571s | 2nd=200 0.015579s
eager run1 | start=2.58s | 1st=200 0.145708s | 2nd=200 0.016802s
lazy  run2 | start=0.21s | 1st=200 2.186814s | 2nd=200 0.015487s
eager run2 | start=2.16s | 1st=200 0.163958s | 2nd=200 0.015420s
lazy  run3 | start=0.22s | 1st=200 1.998263s | 2nd=200 0.017280s
eager run3 | start=2.61s | 1st=200 0.150446s | 2nd=200 0.016556s
```

5. `train_baseline_v1.py` 실행 로그

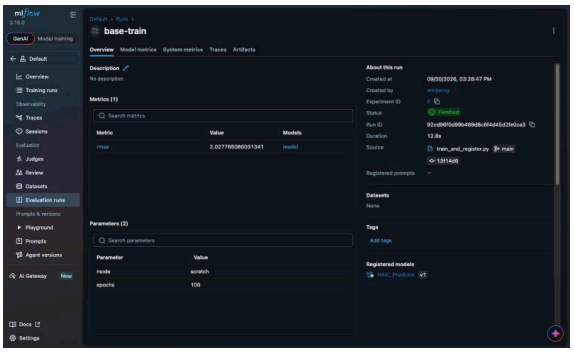
```
$ python scripts/train_baseline_v1.py
scaler fit on 756행 -> serving_app/models/scaler.pkl
baseline v1 RMSE = 2.72  (배포 게이트: $4.00)
```

```
saved -> serving_app/models/haic_v1.keras
real 12.10
```

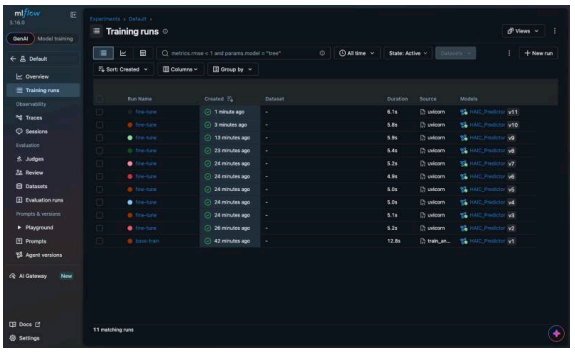
Day2: MLflow 학습, 레지스트리, 컨테이너화

1. `train_and_register.py` 실행 로그와 MLflow 기록

```
$ python serving_app/train_and_register.py
Successfully registered model 'HAIC_Predictor'.
Created version '1' of model 'HAIC_Predictor'.
[GATE PASSED] rmse=2.03 -> HAIC_Predictor v1 promoted to Production
```



[HAIC] base-train run: params(mode=scratch, epochs=100)와 rmse 2.028



[HAIC] run 목록: base 1회 + fine-tune 반복 기록

2. `_load_from_mlflow()` 구현 코드와 MODEL_SOURCE=mlflow 응답

```python
def _load_from_mlflow() -> LoadedModel:
    import mlflow.tensorflow
    from mlflow.tracking import MlflowClient

    keras_model = mlflow.tensorflow.load_model(MLFLOW_MODEL_URI)   # models:/HAIC_Predictor/Production
    scaler = HAICScaler.load(SCALER_PATH)       # 스케일러는 Day1에 fit한 로컬 파일을 그대로
    latest = MlflowClient().get_latest_versions("HAIC_Predictor", stages=["Production"])
    version = f"production-v{latest[0].version}" if latest else "production"   # 교체 여부를 응답으로 확인
    return LoadedModel(keras_model=keras_model, scaler=scaler, version=version)
```

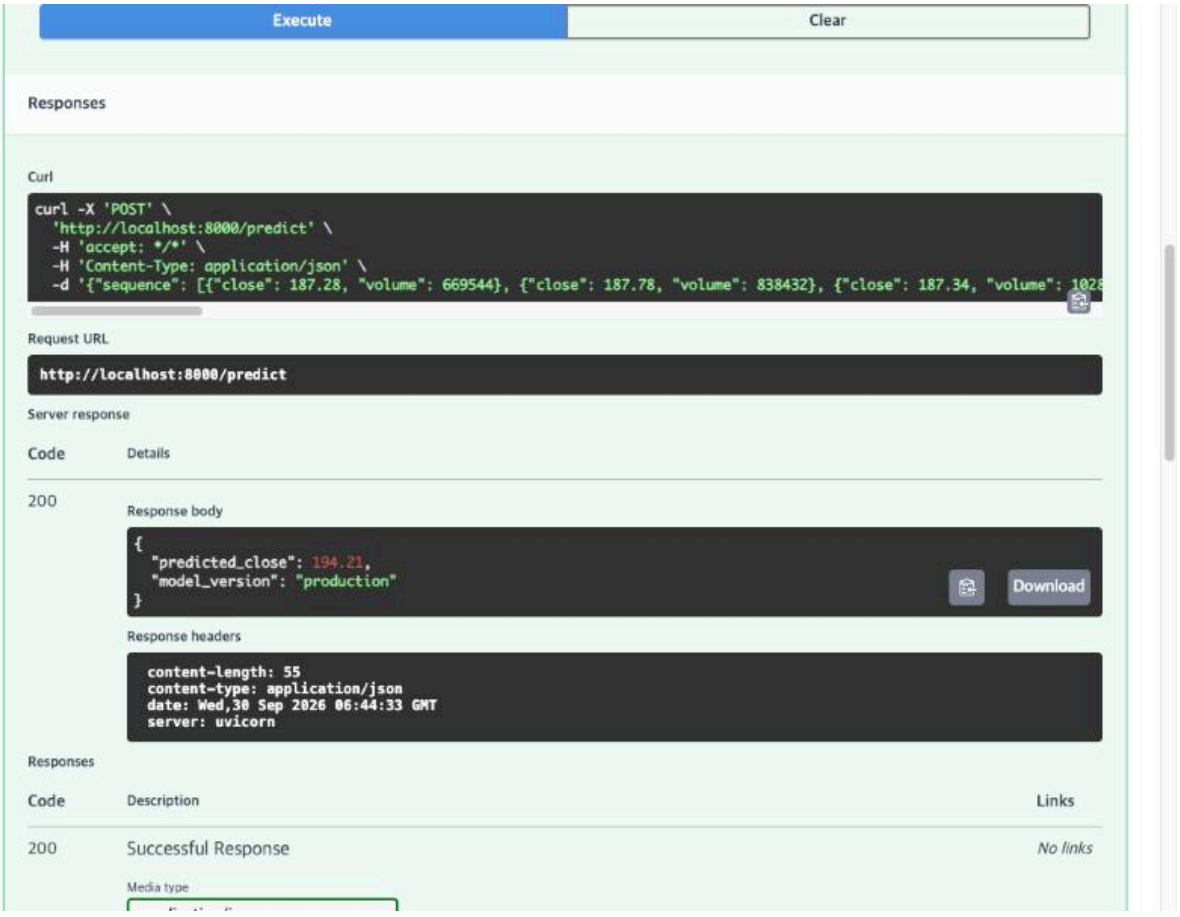
```
## MODEL_SOURCE=local  → {"predicted_close":192.54,"model_version":"v1-local"}
## MODEL_SOURCE=mlflow → {"predicted_close":194.21,"model_version":"production"}  (eager 첫 요청 0.115s)
## main.py, predict.py 수정 없음
```

3. docker compose up --build 로그와 컨테이너 Swagger

통합 가이드는 실습 포트를 8077로 통일했지만, 내 HAIC 실습은 이전 버전 가이드대로 Dockerfile(`EXPOSE 8000`), compose(`8000:8000`), `simulate_drift.py`(`API_URL=localhost:8000`)를 모두 8000으로 맞춰 진행했다. 세 곳의 포

트만 일치하면 절차는 같다(부록 1의 1번 오류가 이 불일치에서 생긴다).

```
$ docker compose -f serving_app/docker-compose.yml up --build      # [HAIC]
#13 [8/8] RUN ... python scripts/train_baseline_v1.py && python serving_app/train_and_register.py
#13 11.67 baseline v1 RMSE = 3.01  (배포 게이트: $4.00)
#13 26.82 [GATE PASSED] rmse=2.03 -> HAIC_Predictor v1 promoted to Production
 Image serving_app-serving-app Built
serving-app-1  | [eager] model loaded in 2.350s at startup
컨테이너 기동 ~ 첫 /health 응답: 3.18s
POST /predict → {"predicted_close":194.21,"model_version":"production"} (0.112s)
컨테이너 재시작 후에도 동일 Production 버전 로드 확인
```



[HAIC] 컨테이너(localhost:8000/docs)에서에서) /predict → model_version production

4. 오늘 실습을 ML 라이프사이클에 매핑한 한 장 요약

라이프사이클 단계	[HAIC]에서 한 일	남는 기록
데이터	IBM 2007~2009 참조 756거래일 CSV 업로드	`data/uploads/`, dataset_sha256
학습	LSTM 3층, 100 epoch, SEED 42	MLflow run (params)
검증	RMSE ≤ $4.00 게이트	`[GATE PASSED/FAILED]`, metrics
배포	Registry Production 승격 → FastAPI가 Production 로드 → Docker	Registry 버전, 컨테이너 로그

라이프사이클 단계	[HAIC]에서 한 일	남는 기록
모니터링	Day3에서 구축 (예측 오차 RMSE)	aiops.log, predictions.jsonl

Day3: 드리프트 감지와 자동 재학습

1. 구현한 4개 함수 [HAIC]

```
# monitoring/drift_detector.py
def compute_rmse(recent_predictions: list[dict]) -> float:
    if not recent_predictions:
        return 0.0
    squared_errors = [(p["actual"] - p["predicted"]) ** 2 for p in recent_
predictions]
    return (sum(squared_errors) / len(squared_errors)) ** 0.5

# routers/predict.py - batch_test() 핵심부
for i in range(len(prices) - SEQ_LEN):
    window = prices[i : i + SEQ_LEN]                          # 길이 20 슬라이
딩 윈도우
    sequence = [{"close": p, "volume": SIMULATED_VOLUME} for p in window]
    pred = model.predict_one(sequence)
    actual = prices[i + SEQ_LEN]                              # 바로 다음 날이
정답
    predictions.append(pred)
    recent_predictions.append({"predicted": pred, "actual": actual})
recent_predictions[:] = recent_predictions[-WINDOW_SIZE:]  # 최근 21건만 유
지

# monitoring/retrain_trigger.py
def check_and_trigger(recent_predictions: list[dict]) -> dict:
    window_rmse = round(compute_rmse(recent_predictions[-WINDOW_SIZE:]),
2)
    if not is_drift(recent_predictions):
        return {"status": "ok", "window_rmse": window_rmse, "n": len(recen
t_predictions)}
    logger.warning(f"[WARN] drift detected - window_rmse={window_rmse:.2f}
> 4.00, triggering retrain")
    logger.info("[INFO] retrain triggered (window=last_21_days)")
    rows = load_rows(latest_upload())[-(WINDOW_SIZE + SEQ_LEN):]   # 최근 2
1 + 20행
    result = fine_tune(rows)
    recent_predictions.clear()          # 옛 모델의 오차로 즉시 재트리거되는 것
을 방지
    if result["promoted"]:
        new_version = model_loader.reload_model().version          # 서빙
캐시 교체 (T3)
        logger.info(f"[OK] new_rmse={result['rmse']:.2f} - production prom
oted: "
```

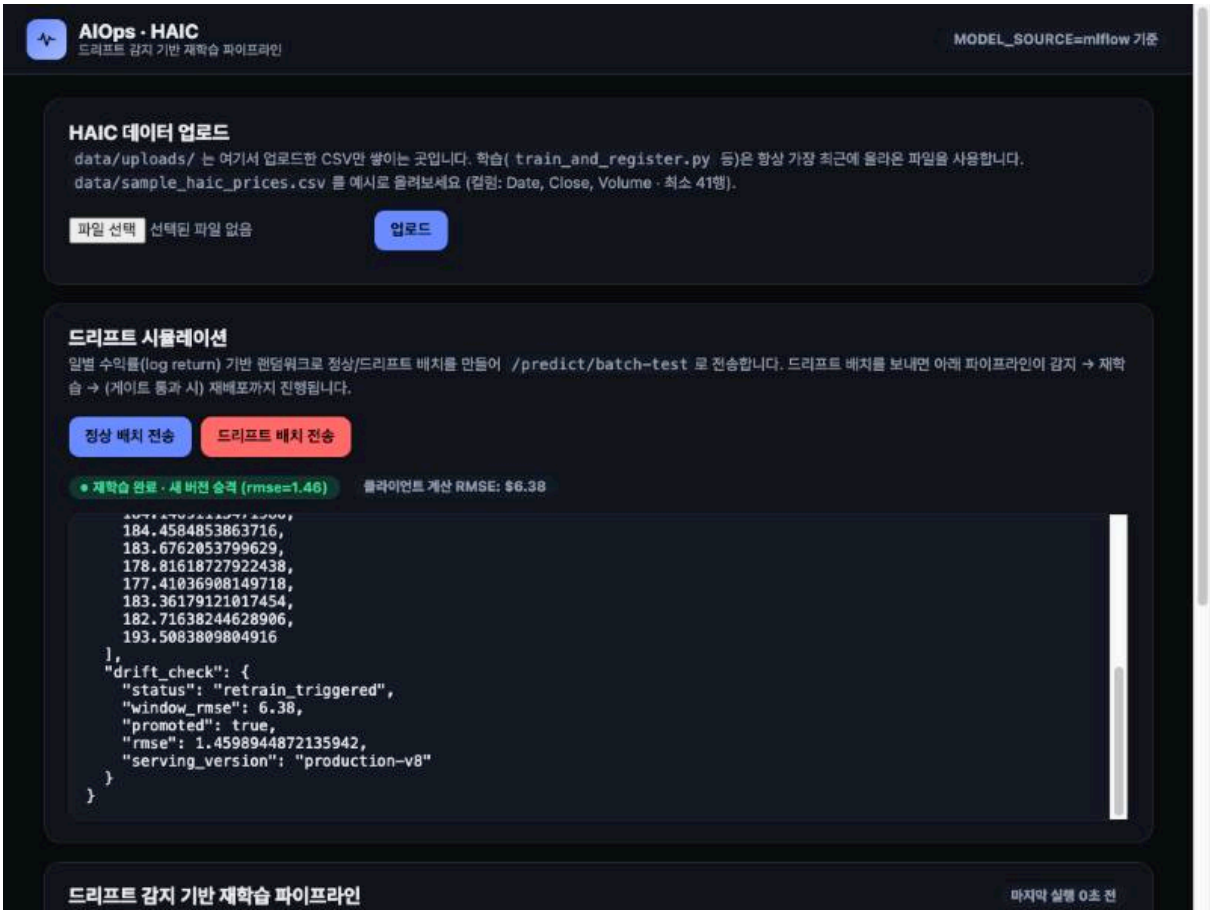
```
                    f"HAIC_Predictor v{result['version']} (serving {new_ve
rsion})")
        return {"status": "retrain_triggered", "window_rmse": window_rmse,
"promoted": True,
                "rmse": result["rmse"], "serving_version": new_version}
    logger.warning(f"[FAIL] new_rmse={result['rmse']:.2f} - gate failed, k
eep current production")
    return {"status": "retrain_triggered", "window_rmse": window_rmse, "pr
omoted": False,
            "rmse": result["rmse"]}

# scripts/simulate_drift.py
def send_batch(prices: np.ndarray, label: str) -> dict:
    resp = requests.post(API_URL, json={"prices": prices.tolist()})
    resp.raise_for_status()
    result = resp.json()
    print(f"[{label}] drift_check = {result['drift_check']}")
    return result
```

단위 확인: `empty → 0.0`, `[3,4 오차] → 3.5355`, `21 × 오차5 → rmse 5.0 drift True`, `21 × 오차3 → drift False`, `20 × 오차5 → drift False (데이터 부족)`.

2. `simulate_drift.py` 실행 결과

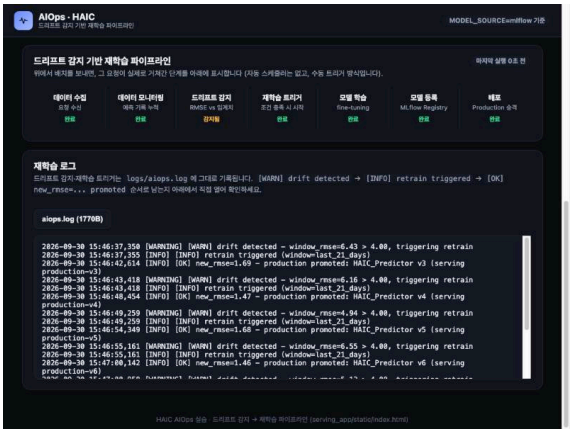
```
# [HAIC]
[normal] drift_check = {'status': 'ok', 'window_rmse': 2.97, 'n': 21}
[drift_injection] drift_check = {'status': 'retrain_triggered', 'window_rmse': 5.13, 'promoted': True,
                                 'rmse': 1.68, 'serving_version': 'production-v7'}
```



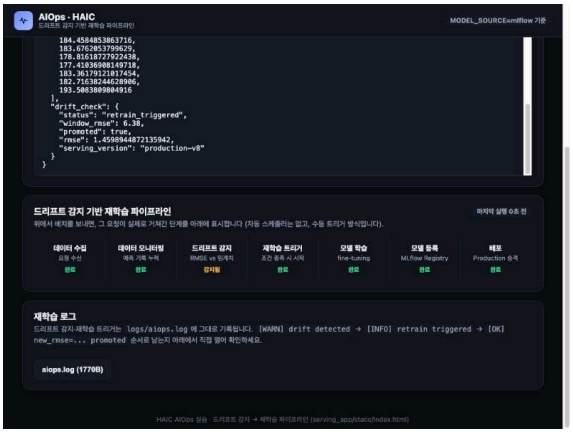
[HAIC] 대시보드 드리프트 배치 결과: retrain_triggered, window_rmse 6.38, serving_version production-v8

3. logs/aiops.log의 [WARN] → [INFO] → [OK]

```
# [HAIC]
2026-09-30 15:47:39 [WARN] drift detected - window_rmse=6.38 > 4.00, triggering retrain
2026-09-30 15:47:39 [INFO] retrain triggered (window=last_21_days)
2026-09-30 15:47:44 [OK] new_rmse=1.46 - production promoted: HAIC_Predictor v8 (serving production-v8)
```



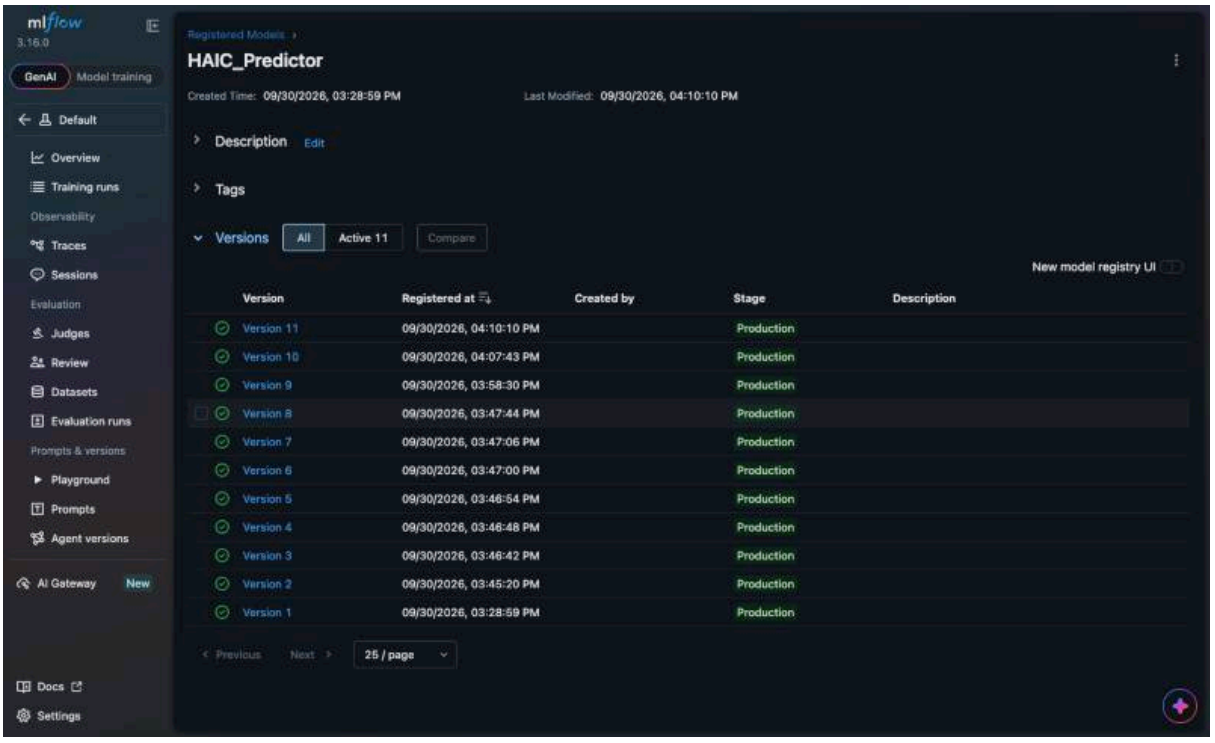
[HAIC] 대시보드 재학습 로그 패널



[HAIC] 재학습 파이프라인 단계 시각화

4. 재학습 후 /predict가 새 Production 버전으로 응답하는가

```
[HAIC]      POST /predict → {"predicted_close":195.16,"model_version":"production-v8"}
```



[HAIC] Registry: 버전이 쌓이는 모습 (Stage가 모두 Production으로 남은 문제는 4장 T4)

마무리

3일 실습을 정리하며
모델을 학습하는 것과 학습한 모델을 서비스에서 계속 잘 동작하게 하는 것은 별도로 고려해야 했다. 이번 실습에서는 서빙, 학습 기록, 배포 게이트, 성능 모니터링, 재학습이 서로 연결되도록 구현하고 확인했다.

가장 크게 바뀐 생각

- 이전에는 RMSE가 낮으면 충분하다고 생각했다. 실습 후에는 **어떤 데이터로 측정한 값인지**(training인지 검증인지), **언제 어떤 범위로 다시 측정하는지**(윈도우와 주기), **기준을 넘으면 무엇을 실행하는지**(알림, 재학습, 게이트)까지 함께 정해야 성능 지표를 운영에 사용할 수 있다고 생각하게 됐다.
- 재학습 로그에 성공이 기록돼도 실제 응답은 기존 모델에서 나올 수 있었다. 이후에는 응답의 버전을 확인하고, Production이 정확히 하나인지도 확인하게 됐다.

다음에 해 보고 싶은 것

- 계측 결과가 몇 시간 뒤에 도착하는 상황에서, 정답을 기다리는 동안 PSI 진단을 선행 경보로 사용하는 정책을 실험해 보고 싶다.
- 재학습이 연속으로 실패하면 담당자에게 확인을 요청하는 에스컬레이션 규칙을 추가하고 싶다. LangGraph로 감지 → 진단 → 재학습/호출 분기를 노드로 구성하는 것도 해 보고 싶다.

- 모니터링 상태를 프로세스 메모리 대신 Redis 같은 공유 저장소에 저장해 다중 워커를 사용하거나 서버를 재시작해도 판정 윈도우를 유지하고 싶다.

서브노트 내용 외 개인 학습 노트

> 아래는 공부하면서 따로 남기고 싶었던 내용이다.

6. 빈칸 채우기 실습: 헷갈렸던 부분

교수님은 **잘못 채워도 에러 없이 결과만 달라질 수 있는 부분에 빈칸을 두었다**고 설명하셨다. 실제로 처음 생각한 값을 넣어도 실행 자체는 되는 경우가 많았다. 그래서 헷갈리는 후보를 하나씩 넣어 보고 결과를 비교했다.

헷갈렸던 다섯 군데

빈칸 1 `model.fit(___, ___)` : 정답은 원래 단위 그대로 넣는 게 자연스럽다고 생각해 처음엔 `y_train`을 넣었다.

- 실행하자 `ValueError: Unrecognized data type: x=[[[0.41...` 에러가 발생했다. 메시지에 x가 표시돼 입력이 문제인 줄 알았지만, 확인해 보니 x는 numpy 배열이고 y만 파이썬 리스트여서 Keras가 둘을 한 묶음으로 해석하지 못한 것이었다.
- y를 배열로 바꾸고 다시 실행하니 에러 없이 끝났지만 RMSE는 **13,920.38**이었다. STEP 7에서는 모델 출력에 `inverse_close`를 적용하는데, 이미 달러 단위의 값을 출력하도록 학습한 결과에 다시 변환을 적용해 값이 커진 것이었다.
- STEP 5에서 0~1로 변환한 `y_train_scaled`를 넣는 것이 맞았다. 이 값으로 다시 학습하니 **RMSE 2.41**이 나왔다.
- 시험 데이터까지 포함한 `fit(X, y 전체)`도 실행해 보니 RMSE 2.49로, 수치만 보면 문제가 없어 보였다. **데이터 누수는 RMSE만으로 확인하기 어렵다**는 것을 알게 됐고, 학습과 평가 데이터를 나누는 코드부터 봐야 한다고 느꼈다.

빈칸 2 (입력 변환): 바로 아래 주석의 "종가 하나를 0~1로"만 보고 `scale_close`를 넣었다.

- `TypeError: scale_close() takes 2 positional arguments but 3 were given` 에러가 발생했다. 함수에 종가와 거래량 두 개를 전달하고 있다는 점을 확인한 뒤, 학습 코드의 `build_sequences`에서 사용하는 변환 함수를 찾아봤다. 학습에서는 `transform_point`를 사용하고 있었다.
- scale_close를 넣었을 때는 에러로 잘못된 입력 변환을 알 수 있었다. 반면 변환을 아예 빼면 에러 없이 **146.03**이 반환됐다(정답 189.66, 마지막 실제 종가 192.71). 그래서 실행 여부만 확인하지 말고 **학습 때와 같은 변환 함수를 사용했는지** 확인해야 한다고 생각했다.

빈칸 6 (슬라이딩 윈도우): 입력 윈도우의 마지막 날 가격을 예측하는 것으로 잘못 이해해 처음에는 `actual = prices[i + SEQ_LEN - 1]`로 작성했다.

- 정상 배치(41개)에서 RMSE가 **0.34**로 나와 예상보다 낮았다. 정답 인덱스인 `i + SEQ_LEN`을 사용했을 때는 **2.36**이었다.
- 파이썬 슬라이싱을 다시 확인하니 `prices[a:b]` 는 b를 포함하지 않았다. `prices[i : i + SEQ_LEN]`의 범위는 i ~ i+19이므로 다음 날 정답은 `i + SEQ_LEN`에 있다. 한 칸 앞의 값을 정답으로 사용하면 모델이 입력에서 이미 본 값으로 평가하게 된다.
- 이 인덱스를 잘못 설정하면 드리프트 배치에서도 RMSE가 낮게 나와 **드리프트를 놓칠 수 있다.** 예상보다 결과가 너무 좋을 때는 정답으로 사용한 값이 올바른지 먼저 확인해야겠다고 생각했다.

빈칸 7 (RMSE): 처음에는 평균적으로 몇 달러 차이가 나는지 계산하면 된다고 생각해 오차를 그대로 평균 내려고 했다.

- docstring 예시(+2, -2)를 계산하니 **0.0**이 나왔다. 반대 방향의 오차가 서로 상쇄되어, 두 번 모두 2달러씩 틀렸는데도 오차가 없는 것처럼 계산됐다.
- 제곱근을 빼면 **4.0**이 나온다. 단위가 달러²라서 $4.00 기준과 비교하면 안 된다.
- 따라서 첫 번째 빈칸에는 `실제 - 예측`, 세 번째 빈칸에는 `합 ÷ 개수`를 넣었다. `예측 - 실제`로 순서를 바꾸어도 결과는 2.0으로 같았다. 오차를 제곱하므로 부호가 결과에 영향을 주지 않았다.

빈칸 5 (MLflow 모델의 스케일러): 모델을 MLflow에서 가져오니 스케일러도 "최신 데이터로 새로 맞춰야 하나?" 싶었다.

- 최근 41행으로 스케일러를 새로 fit한 뒤 실행하니 예측값이 **192.47**로 바뀌었다. 마지막 실제 종가(192.71)에 더 가까워 결과만 보면 이 방법이 맞는 것처럼 보였다.
- `train_and_register.py`를 확인하니 Day2 모델도 Day1의 `scaler.pkl`로 변환한 데이터로 학습했다. 기존 가중치와 같은 변환 기준을 사용해야 하므로 `HAICScaler.load(SCALER_PATH)`로 불러오는 것이 맞았다.
- **예측값이 실제값에 가까워 보인다는 이유만으로 맞는 구현이라고 판단할 수는 없었다.** 학습과 서빙에서 같은 스케일러를 사용했는지 확인하는 것이 판단 기준이었다.

그 밖에 확인한 것

- **빈칸 8**: 3건 중 1건의 예측이 15달러 차이가 나도 RMSE는 8.66이었다. 최소 건수 조건이 없으면 서버 시작 직후에도 재학습이 실행될 수 있었다. 정답 조건을 넣으면 `False`를 반환해 판단을 보류했다.
- **빈칸 9**: 21행만 사용하면 `build_sequences`로 만들어지는 시퀀스는 **1개**였다. `21 + SEQ_LEN`(41행)을 사용하면 **21개**가 만들어졌다. 필요한 행 수는 예측할 정답 개수에 입력 윈도우 길이를 더해 계산해야 했다.
- **빈칸 4 (예상과 달랐던 점)**: 첫 로딩은 3.03초였지만 조건 없이 다시 불러오는 데는 0.021초밖에 걸리지 않았다. 첫 로딩 시간의 대부분은 TensorFlow 초기화에 사용된 것으로 보인다. 그래도 요청마다 모델 객체를 새로 생성하면 메모리와 동시 요청 처리에 불필요한 비용이 생기므로 `is None` 조건을 두는 것이 맞다고 판단했다.
- **정답을 모두 채운 뒤 실행한 전체 루프**: 정상 배치에서 `ok`와 예측 21개를 확인했고, 드리프트 배치에서는 `[WARN] → [INFO] → [OK] new_rmse=1.68 promoted v13`까지 실행됐다. 하지만 재학습 직후 /predict는 여전히 `193.37, production`을 반환했다. 빈칸을 채운 것과 별개로 서빙 캐시를 교체하는 작업이 필요했다(4-1의 T3와 같은 문제).

실습을 하면서 알게 된 점
에러가 발생한 경우에는 잘못 넣은 값을 비교적 쉽게 찾을 수 있었다. 오히려 에러 없이 실행됐는데 결과가 너무 좋거나(0.34, 0.0), 실제값에 가까워 보이거나(192.47), 비정상적으로 큰(13,920) 경우에 판단하기 어려웠다. 이후에는 빈칸을 채우기 전에 학습에서 사용한 함수와 데이터, 값의 단위, 정답 인덱스가 입력에 포함된 값인지부터 확인하게 됐다.

7. 교수님이 강조한 것, 그리고 내 생각

수업 중 교수님이 강조하신 말씀을 듣고, 실습하면서 깨달은 점을 정리했습니다.

1. "데이터는 업무의 결과다. 모델을 놔두면 3개월 뒤 안 맞는다고 전화가 온다."

> 업무는 계속 바뀌고, 학습시킨 것과 다른 패턴의 데이터가 들어오면 성능이 떨어진다. 한 번 학습시켜 두면 끝이 아니라 지속적으로 성능이 유지되도록 운영해야 한다.

업무가 얼마나 자주 달라지는지에 따라 모델을 다시 확인해야 하는 주기도 달라진다고 이해했다. EtchGuard에서 재학습 데이터를 최근 7일로 정할 때도 식각 공정 상태가 어느 주기로 바뀌는지 고려했다. 교수님이 말씀하신 계절 변화도 프로젝트에 적용해 보았다. 계절에 따른 변화는 매년 반복될 수 있는 정상 변화이므로, 이를 모두 드리프트로 판단하지 않도록 같은 달의 기준 분포를 사용했다.

2. "학습과 서빙은 다르다. 운영이 걸리는 순간 성능, 속도, 사용자 수가 다 영향을 준다."

> 로컬에서는 잘 돌아가는데 운영에 띄웠더니 특정 시간대에 사용자가 몰려 서비스가 안 된다. 예측은 못 해도 베이스라인은 정할 수 있다.

실습에서 Lazy와 Eager를 각각 실행해 보니 교수님이 말씀하신 학습과 서빙의 차이를 이해할 수 있었다. RMSE 2.03은 학습 결과로 정해졌지만, 첫 요청에 2.31초가 걸리는지 0.15초가 걸리는지는 서빙에서 모델을 언제 로드하는지에 따라 달라졌다. 같은 모델을 사용해도 사용자가 기다리는 시간은 약 15배 차이가 났다. 교수님이 말씀하신 베이스라인은 EtchGuard의 SLO로 적용해 첫 요청 1초 미만, p95 1초 미만으로 정했다.

3. "API는 계약이다. 규격을 정하지 않으면 오류가 오염돼서 전파된다."

> 음식점에 전화 주문할 때 손님이 무슨 말을 할지 모르면 사장님이 계속 지켜보며 적응해야 한다. 규격이 있으면 누르면 끝난다. 중간에 터지면 어디서 문제가 생겼는지 찾기 어렵다.

교수님의 주문 비유를 듣고 입력 형식을 미리 정해야 하는 이유를 이해했다. EtchGuard에서는 허용하는 입력뿐 아니라 받으면 안 되는 필드도 정했다. 예측 요청에 정답 필드인 `anomaly_score`가 포함되면 422로 거절하도록 했다. 이렇게 API에서부터 정답 데이터가 입력에 섞이는 것을 막아 학습 시 정답을 보고 맞히는 누설을 방지했다.

4. "로그를 어떻게 남길지가 중요하다. 다만 uvicorn 포트 번호 같은 로그는 파고들 필요 없다."

> 의미를 부여해야 할 로그는 따로 있다. 데이터 드리프트 이력에 남기는 로그를 분석하라.

로그를 누가 사용하는지에 따라 남겨야 할 내용도 다르다고 정리했다. uvicorn 접근 로그는 사람이 요청과 문제를 추적할 때 확인한다. `[WARN] → [INFO] → [OK]` 태그가 있는 aiops.log는 대시보드가 파싱하므로, 태그 문자열을 바꾸면 연결된 처리도 영향을 받을 수 있다. 그래서 로그 형식도 유지해야 할 규칙이라고 생각했다.

5. "Lazy와 Eager는 상황에 따라 다르다. 비동기는 느린 게 아니다."

> 서비스를 띄운 뒤 어떤 모델을 쓸지 나중에 정한다면 Lazy, 사용자가 들어오는 순간 로딩되면 이상하니 그런 경우엔 미리 띄운다.

3-2의 1번에서 실행 시간을 비교해 보니 Lazy와 Eager 중 하나가 항상 맞는 것은 아니고, 상황에 따라 선택해야 한다는 말을 이해할 수 있었다. 비동기 처리는 조 프로젝트에서 재학습을 백그라운드 스레드로 분리하면서 체감했다. 수 초가 걸리는 재학습을 요청 처리 안에서 동기로 실행하면 드리프트 배치 응답도 그만큼 늦어진다. 그래서 먼저 `retrain_triggered` 응답을 반환하고, 진행 상황은 `/monitoring/status`로 확인하도록 했다.

6. "반드시 기억할 키워드 두 개: 에이전트, 그리고 와이어링."

> 사람이 지연과 에러를 계속 지켜볼 수는 없다. 운영에는 반드시 에이전트가 들어온다. AIOps는 지금까지 배운 것을 와이어(선)로 연결해 실전에 적용하는 과목이다.

우리 `retrain_trigger.py`는 감지 → 판단 → 행동(재학습) → 검증을 수행하지만, 판단은 if문으로 작성한 규칙에 따른다. 그래서 에이전트의 역할을 간단한 모듈로 구현한 것으로 이해했다. 다음에는 LangGraph로 드리프트 감지, 원인 센서 진단, 재학습 또는 엔지니어 호출을 각각 노드로 구성해 보고 싶다. 여기에 LLM이 aiops.log와 원인 센서 정보를 읽고 점검 보고서를 작성하는 기능을 붙이면, 교수님이 말씀하신 예측형 AI와 생성형 AI의 결합을 구현할 수 있을 것 같다.

7. "AI는 도깨비 방망이가 아니라 거울이다. AI 없이 풀리는 문제에는 AI를 쓰지 마라."

> 결과를 검증하고 이해관계자의 가치에 맞게 동작하는지 보증하는 것은 사람의 몫이다.

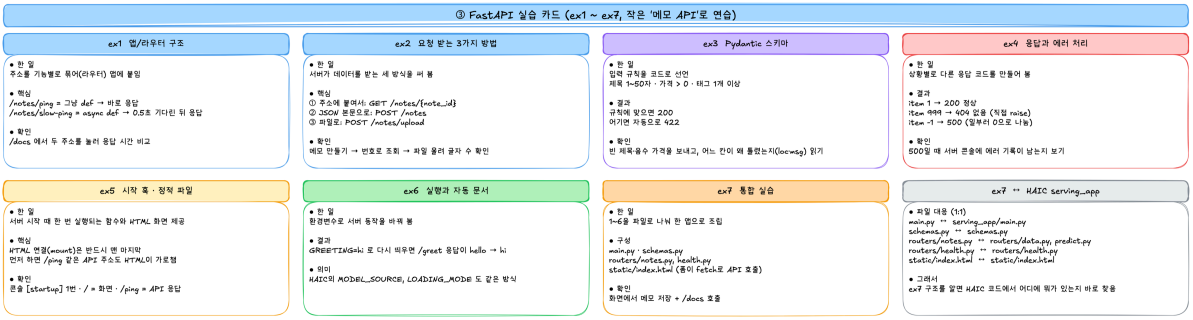
조 프로젝트에서 내가 맡은 모델 평가에서는 LSTM을 직전값을 그대로 쓰는 단순 규칙과 계속 비교했다. 정상 구간에서는 LSTM의 RMSE가 17.5% 더 낮았지만, 드리프트 구간에서는 8.07로 단순 규칙(3.57)보다 나빴다. AI를 사용하더라도 단순한 방법보다 성능이 떨어지는 시점을 확인할 수 있어야 한다고 느꼈다. 이 노트를 쓸 때 AI의 도움을 받았지만, 수치는 모두 실행 로그와 맞춰 보았다.

8. "도메인을 바꿔 포트폴리오로 만들어라. 취미 수준이라는 말을 들으면 안 된다. AI에게 맥락 없이 물으면 엉뚱한 답을 한다."

교수님의 말씀을 듣고, 제공된 파이프라인을 현업에 맞는 도메인으로 바꾸어 기획하는 것이 우리 과제라고 이해했다. 수업용 주가 데이터 그대로보다는 실제 사용 상황을 정해 적용할 필요가 있었다. 우리 조가 반도체 식각을 선택한 뒤에는 모델 구조보다 계절 변화와 실제 이상을 어떻게 구분할지, 오경보를 어느 정도까지 허용할지와 같은 운영 기준을 오래 논의했다. 이 과정도 기록해 두면 나중에 AI에게 질문할 때 필요한 맥락을 전달할 수 있다고 생각했다.

8. FastAPI 문법 정리 (실습 1 ~ 7)

교수님은 FastAPI 문법 중 모델 서빙에 필요한 부분만 6개로 나눠 설명하고, 7번 실습에서 하나로 합치셨다. 각 실습 서버를 다시 띄워 Swagger에서 호출해 보았다.



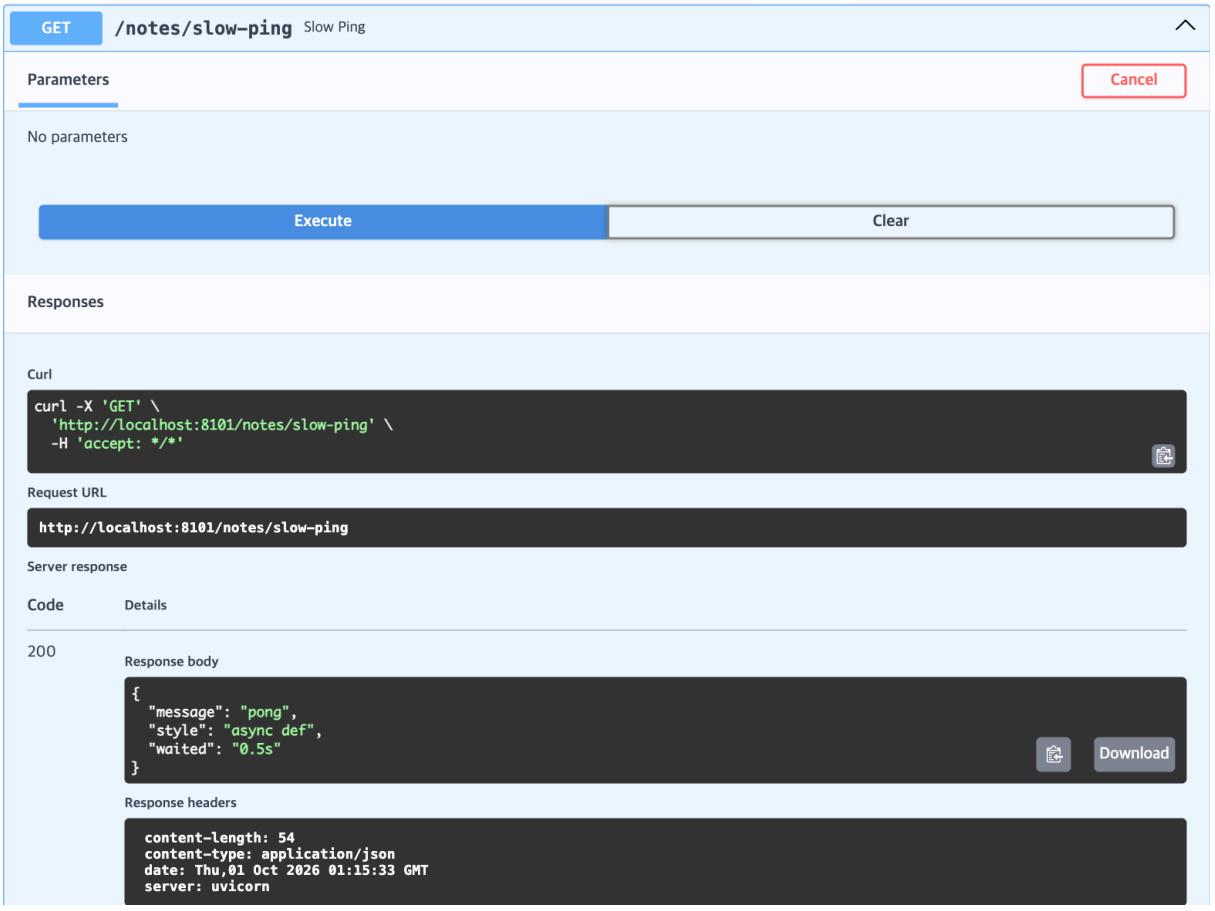
실습 1. 앱과 라우터 구조

```
app = FastAPI(title="1. 앱/라우터 구조 실습")
notes_router = APIRouter(prefix="/notes")
system_router = APIRouter()

@notes_router.get("/slow-ping")
```

```python
async def slow_ping():
    await asyncio.sleep(0.5)          # 기다리는 동안 이벤트 루프는 다른 요청을
받는다
    return {"message": "pong", "style": "async def", "waited": "0.5s"}

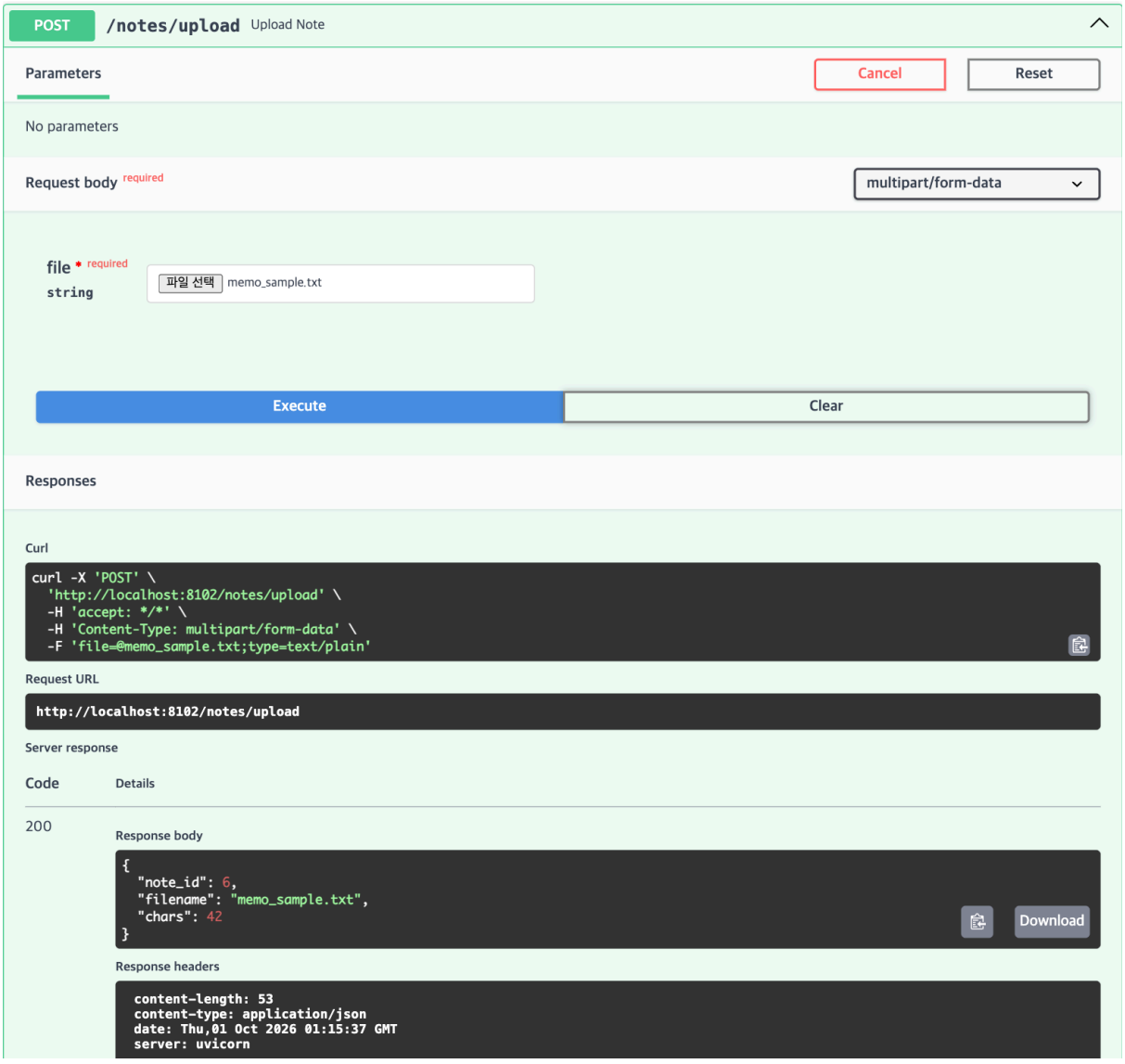
app.include_router(notes_router)      # 등록 순서가 경로 매칭 순서가 된다
app.include_router(system_router)
```



/notes/slow-ping 호출: async def + await로 0.5초 뒤 응답

정리: 경로는 기능별 `APIRouter` 파일로 나누고, main에서 `include_router`로 모으면 하나의 앱이 된다.

실습 2. 요청을 받는 3가지 방법



POST /notes/upload: 텍스트 파일 업로드 후 글자 수(chars) 반환

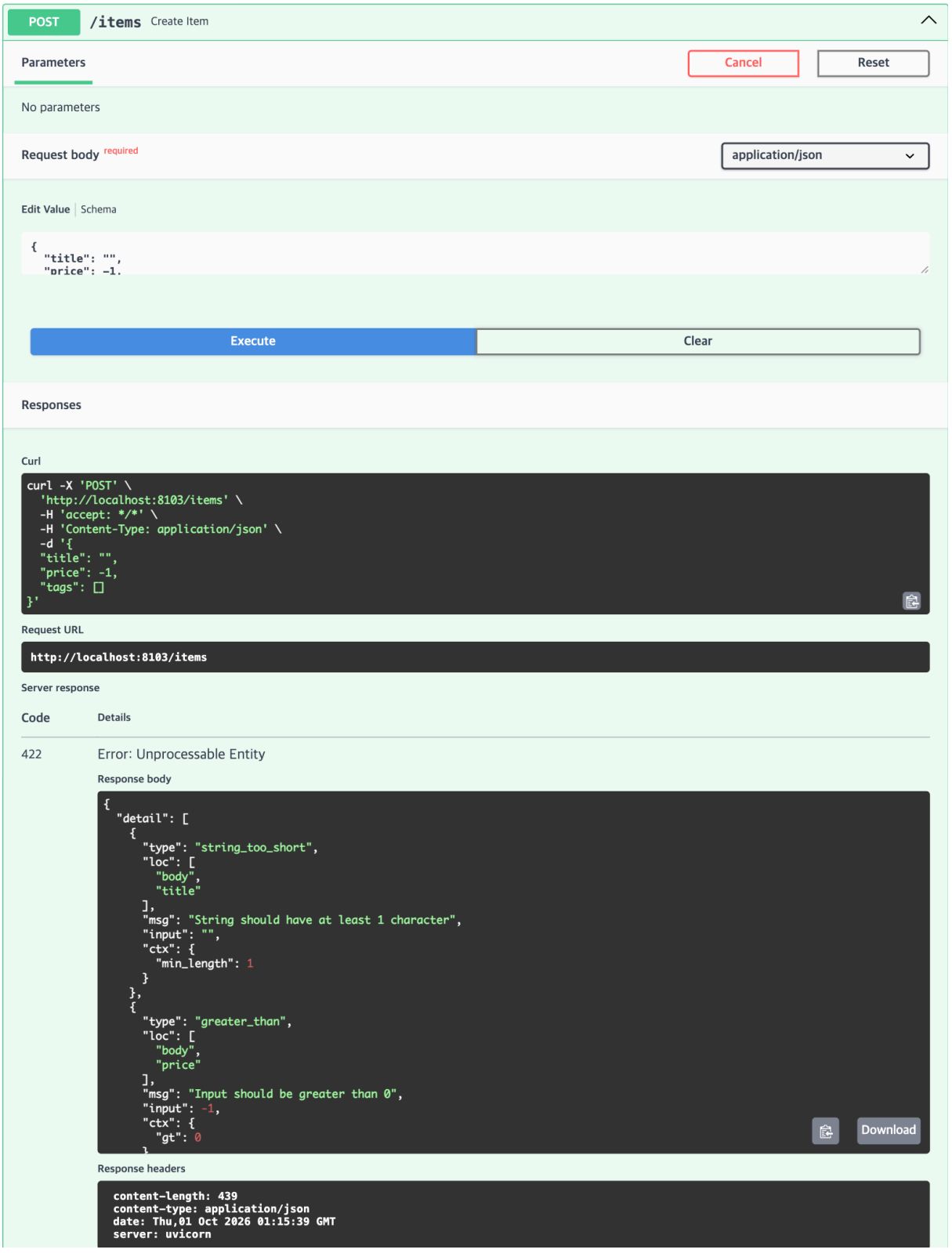
정리: 식별자는 경로로, 구조화된 입력은 바디로, 큰 원본 데이터는 파일로 받는다. HAIC의 CSV 업로드가 파일 방식이다.

실습 3. Pydantic 스키마

```python
class Tag(BaseModel):
    name: str = Field(..., min_length=1)

class ItemCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=50)
    price: float = Field(..., gt=0)
    tags: list[Tag] = Field(..., min_length=1)   # 중첩 모델

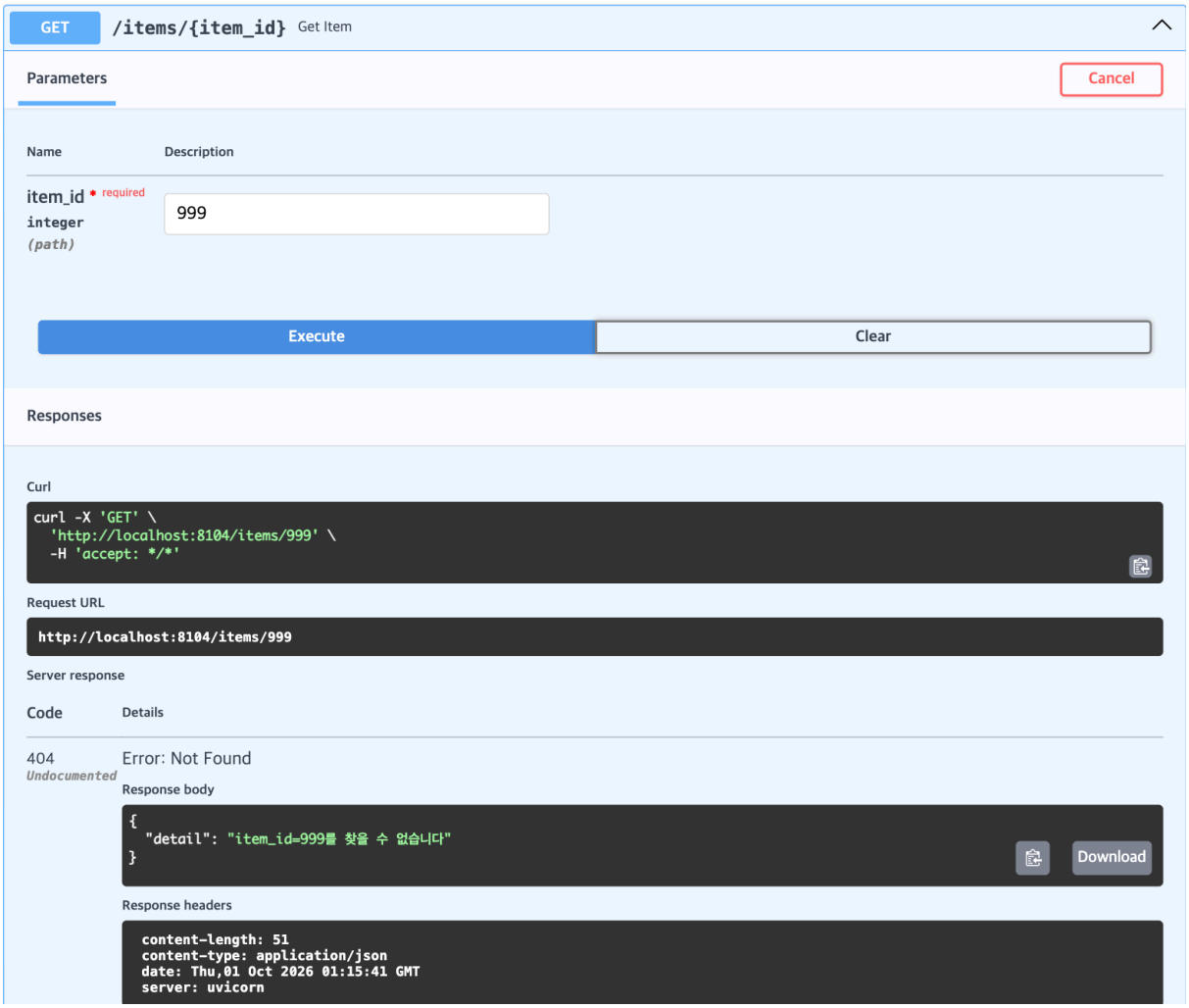
@app.post("/items", response_model=ItemResponse)  # 응답도 같은 방식으로 검증
def create_item(item: ItemCreate): ...
```



잘못된 값(title 빈 문자열, price -1, tags 빈 리스트): 422와 함께 loc와 msg로 어느 필드가 왜 틀렸는지 반환

정리: 잘못된 값 세 개를 보내면 422 응답 하나에 오류 3개(`string_too_short`, `greater_than`, `too_short`)가 같이 온다. HAIC `schemas.py`의 시퀀스 길이 20, `close > 0` 검증도 같은 문법이다.

실습 4. 응답과 에러 처리



item_id=999: HTTPException으로 의도한 404와 이유(detail)

```
# 같은 순간 서버 콘솔 (클라이언트는 이 내용을 볼 수 없다)
    return 1 / 0  # ZeroDivisionError -> 500 Internal Server Error
ZeroDivisionError: division by zero
```

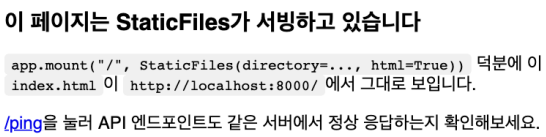
정리: 예상한 실패는 `HTTPException`으로 상태코드와 이유를 정해 주고, 예상 못 한 예외는 500이 되므로 서버 로그로 원인을 찾아야 한다.

실습 5. 생명주기와 정적 파일

```python
@app.get("/ping")
def ping(): return {"message": "pong"}

@app.on_event("startup")
def startup(): print("[startup] 서버 준비 완료")     # 기동 시 딱 한 번 → Eager 로딩 자리

app.mount("/", StaticFiles(directory=_STATIC_DIR, html=True), name="static")  # 반드시 마지막
```

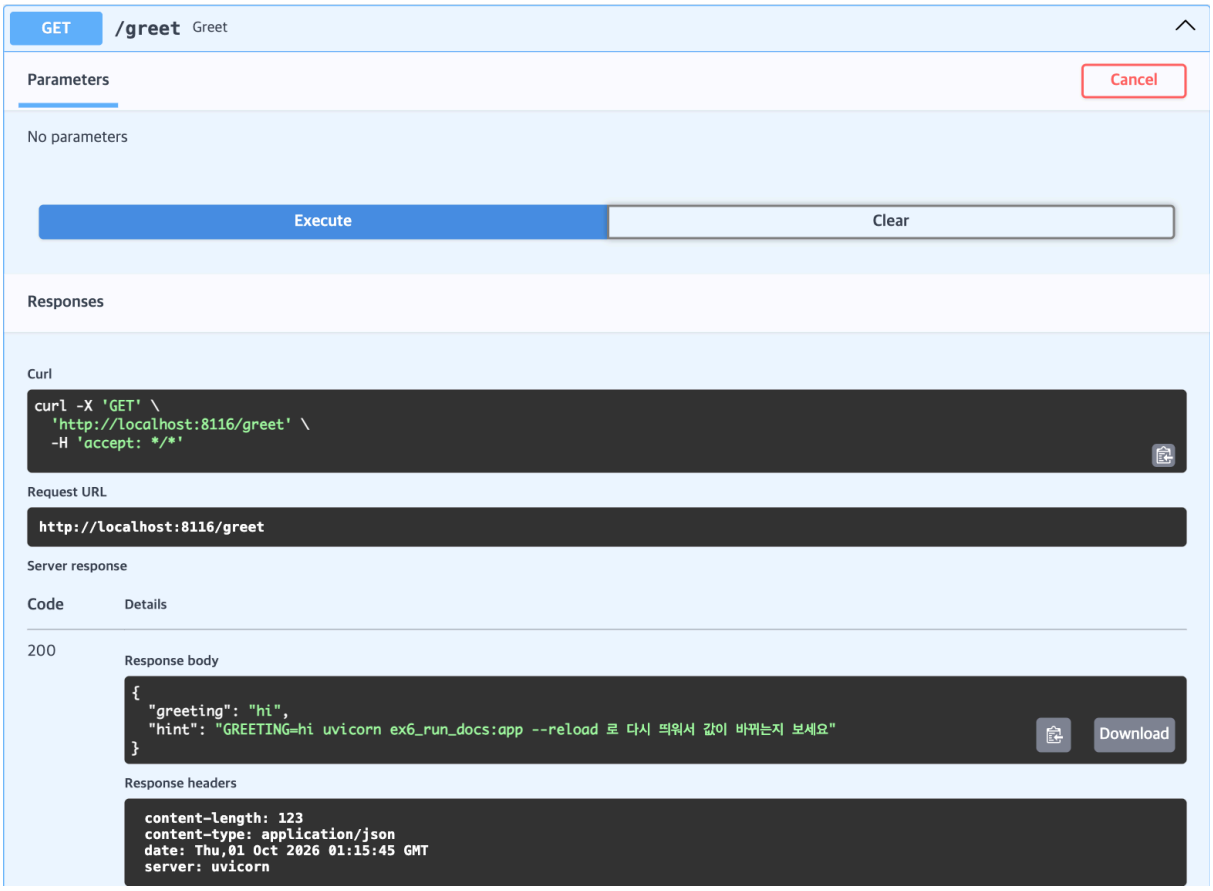


localhost:8105 접속 시 static/index.html이 같은 서버에서 서빙됨

정리: `/`에 정적 파일을 mount하면 모든 경로를 가져가므로 API 라우터를 먼저 등록해야 한다. startup 함수는 기동 때 한 번만 실행되고, Eager 로딩이 바로 이 자리에 들어간다.

실습 6. 실행과 자동 문서 (환경변수 주입)

```
uvicorn ex6_run_docs:app --port 8106              # 기본값
GREETING=hi uvicorn ex6_run_docs:app --port 8116  # 코드 수정 없이 동작 변경
```

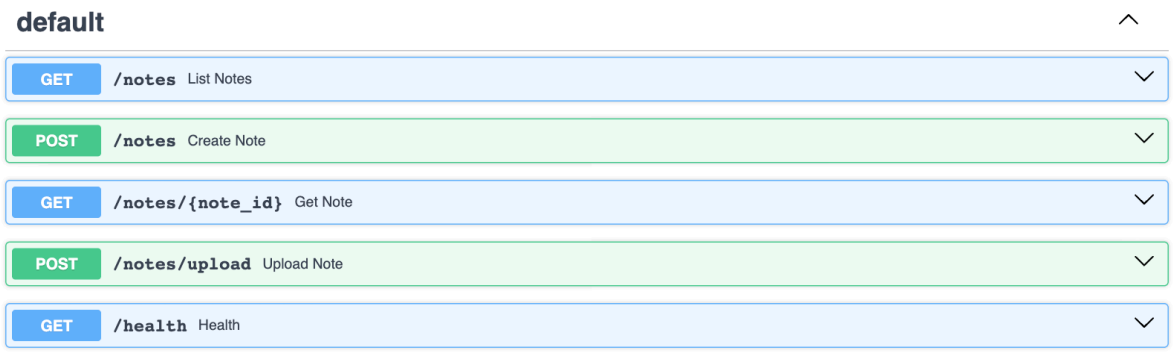


GREETING=hi로 실행: greeting = hi

정리: 코드를 고치지 않고 환경변수로 동작을 바꾸는 방식이 `MODEL_SOURCE`, `LOADING_MODE`와 Dockerfile의 `ENV`에 그대로 쓰인다.

실습 7. 통합 (1~6을 하나로)

```
ex7_integrated/main.py            ↔  serving_app/main.py
ex7_integrated/schemas.py         ↔  serving_app/schemas.py
ex7_integrated/routers/notes.py   ↔  serving_app/routers/data.py, predict.py
ex7_integrated/routers/health.py  ↔  serving_app/routers/health.py
ex7_integrated/static/index.html  ↔  serving_app/static/index.html
```

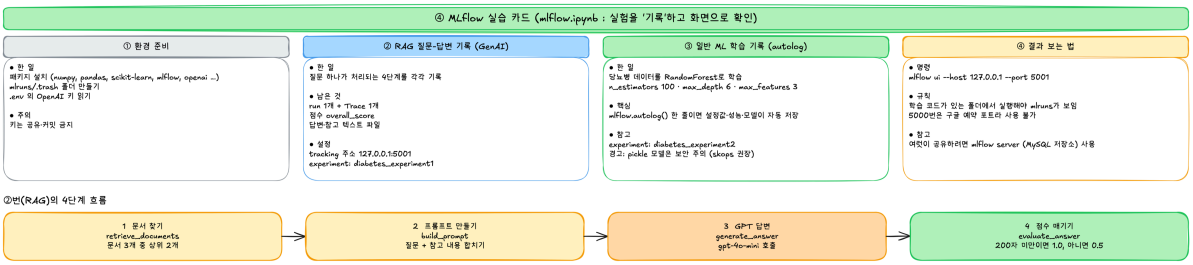


통합 앱의 Swagger: main.py에는 데코레이터가 하나도 없지만 라우터 파일에서 엔드포인트가 들어옴

정리: `main.py`에는 `@app.get`이 하나도 없지만 Swagger에 엔드포인트 5개가 보인다. 라우터 파일을 `include_router`로 붙였기 때문이고, HAIC `serving_app/`도 같은 구조다.

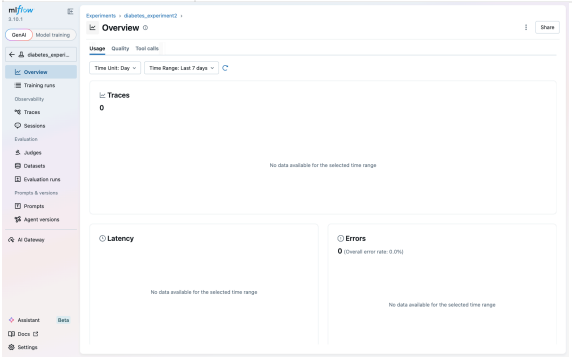
9. MLflow 실습 정리 (두 번째 소스코드 `mlflow.ipynb`)

노트북이 무엇을 기록하는지 카드로 정리했다. 아래쪽은 RAG 질문 하나가 처리되는 4단계다.

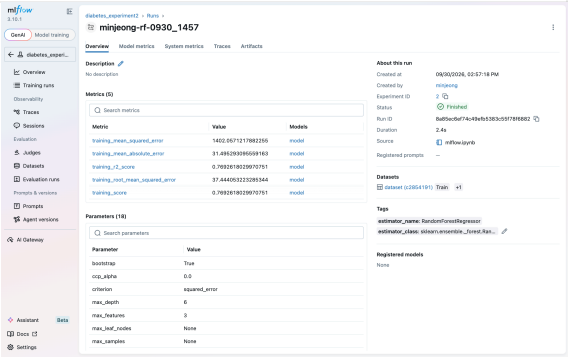


노트북 구성

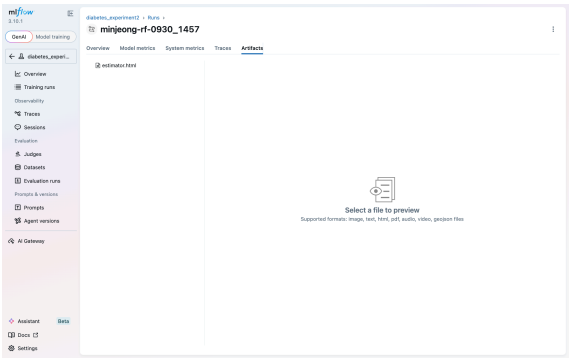
셀	하는 일	핵심 코드
0	패키지 설치, `mlruns/.trash` 생성	`mlflow>=3.9,<3.11`, `openai`, `python-dotenv`
1	GenAI 추적: 간단한 RAG(문서 3개)로 당뇨병 질문에 답하고 OpenAI 호출을 trace로 기록	`mlflow.set_tracking_uri("http://127.0.0.1:5001")`, `mlflow.set_experiment("diabetes_experiment1")`, `mlflow.openai.autolog()`
2	ML 추적: 당뇨병 데이터 RandomForest 회귀를 autolog로 기록	`mlflow.autolog()`, `RandomForestRegressor(n_estimators=100, max_depth=6, max_features=3)`, `test_size=0.2, random_state=42`
3	서버 버전과 로컬 UI 차이 설명	`mlflow server --backend-store-uri mysql+pymysql://...` vs `mlflow ui --port 5001` (5000번은 예약 포트라 피함)



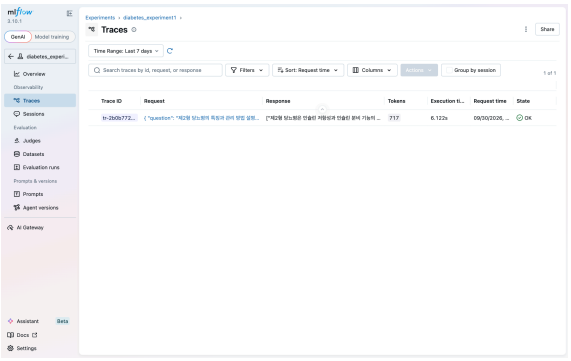
diabetes_experiment2의 run 목록



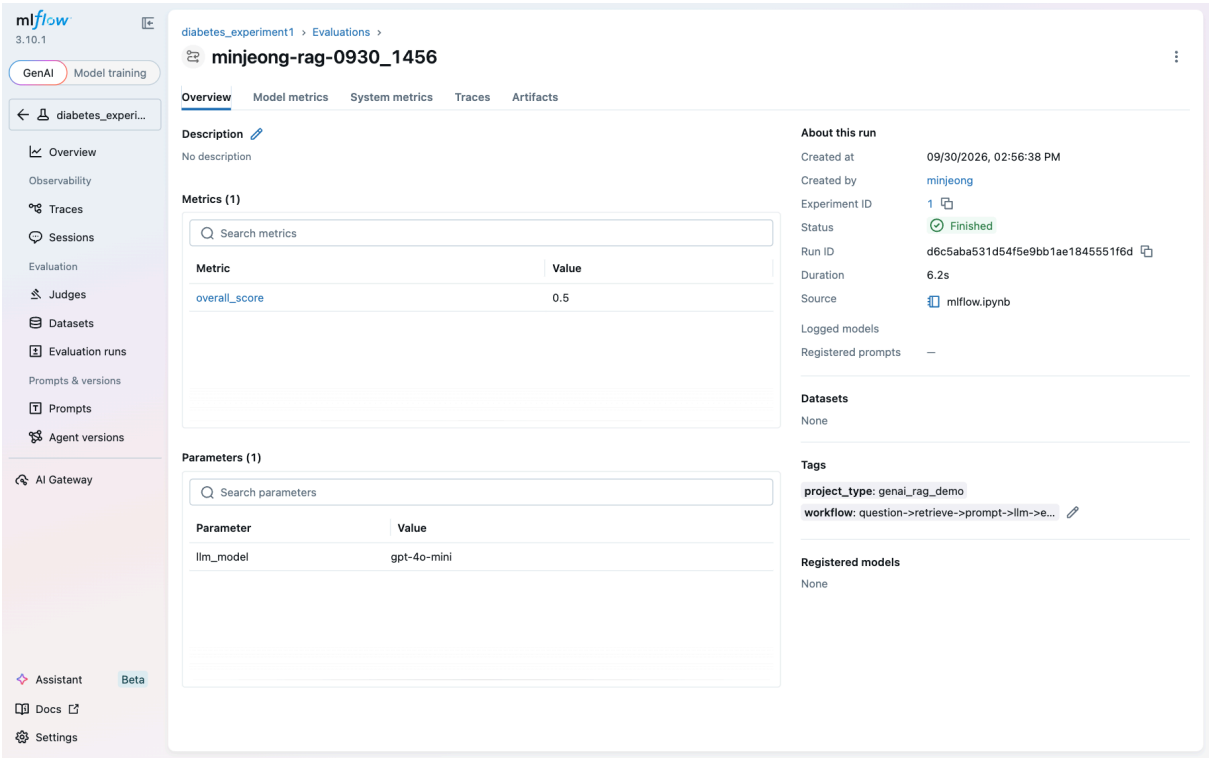
RF run 상세: autolog가 남긴 지표 5개와 파라미터 18개, Registered models는 None



RF run의 Artifacts: 모델 파일, 환경(conda, requirements), signature가 자동 저장



GenAI 실험의 Trace: 질문, 응답, 토큰 717개, 실행 6.122초, 상태 OK



GenAI run: llm_model=gpt-4o-mini 파라미터와 overall_score 0.5 지표

MLflow 기록에서 확인한 내용

autolog가 기록한 지표는 모두 `training_*` 였다. MLflow에서 `training_r2_score 0.769`, `training_root_mean_squared_error 37.44`, `training_mean_squared_error 1402.06`을 확인했다. 노트북은 `rf.predict(X_test)`를 호출했지만 테스트 점수를 별도로 계산하고 기록하지 않아 **학습 데이터의 점수만** 남았다. 이 값으로 배포를 판단하면 성능을 과대평가한 모델이 통과할 수 있다고 생각했다. 그래서 HAIC의 `train_and_register.py`와 EtchGuard의 `train.py`에서는 autolog만 사용하지 않고, 학습에 사용하지 않은 검증셋의 RMSE를 따로 `mlflow.log_metric("rmse", ...)`로 기록한 뒤 **그 값으로 게이트를 판정한다.**

- **Tracking과 Registry의 차이**: RF run의 Registered models가 None으로 표시됐다. 실험은 기록했지만 모델은 등록하지 않은 상태였다. 배포 대상으로 관리하려면 `mlflow.register_model()`로 이름과 버전을 등록해야 한다.

- **LLM 호출 기록**: trace에서 토큰 수와 실행 시간을 확인할 수 있었다. 교수님이 말씀하신 드리프트 감지 후 LLM 보고서 생성 기능을 추가한다면, LLM 호출의 토큰 비용과 응답 지연도 함께 모니터링해야 한다고 생각했다.
- **실험 이름 설정**: `set_experiment()`는 ID가 아니라 이름을 받고, 해당 이름이 없으면 실험을 새로 생성한다. 실행 전 `mlflow.active_run()`으로 진행 중인 run을 확인하고 끝내면 중첩 run 오류를 방지할 수 있다.

노트북 → HAIC → EtchGuard로 가면서 바뀐 것

항목	mlflow.ipynb	[HAIC] `train_and_register.py`	[EtchGuard] train.register_bundle()
기록 방식	autolog (자동)	수동 `log_param`, `log_metric("rmse")`	수동, 구간별(validation, test, drift, champion, golden) 지표 + `evaluation.json`
모델 형식	sklearn	tensorflow (Keras)	pyfunc 래퍼: 모델 + scaler.json + 기준 분포를 하나의 artifact로
등록	없음	게이트 통과 시 `HAIC_Predictor` 등록	통과와 실패 모두 등록, `gate_passed` 태그
승격	없음	stage → Production	alias `@Production` (실패 시 이동 안 함)
저장소	추적 서버 5001 (SQLite)	프로젝트 루트 `mlflow.db`	`config.tracking_uri()` 한곳에서 결정, 컨테이너 안에서 빌드 시 생성